

Table of Content

SEMESTER PROJECT: CLOUD-BASED LIBRARY MANAGEMENT SYSTEM	4
1. PROJECT PROPOSAL, EXECUTIVE SUMMARY & PROBLEM DEFINITION	4
1.1 Executive Summary	4
1.2 Introduction	4
1.3 Problem Statement	4
1.4 Aim & Objectives	4-5
2. AWS ARCHITECTURE DESIGN & IMPLEMENTATION	5
2.1 AWS Architecture Description	6
2.2 Implementation Steps	6-7
2.3 Security & IAM	7
3. SCREENSHOTS OF THE IMPLEMENTATION	8-34
PHASE 1: LAB ENVIRONMENT & AUTHENTICATION	8
PHASE 2: NETWORK INFRASTRUCTURE (VPC)	9
PHASE 3: SECURITY & ACCESS CONTROL	16
PHASE 4: DATABASE DEPLOYMENT (RDS)	18
PHASE 5: OBJECT STORAGE	20
PHASE 6: CONTAINER REGISTRY (ECR)	23
PHASE 7: LOAD BALANCING (ALB)	24
PHASE 8: ORCHESTRATION (ECS)	26
PHASE 9: FINAL APPLICATION OUTPUT	27
PHASE 10: MONITORING & OBSERVABILITY	32
3. REFERENCES	34

Table of Figures

SEMESTER PROJECT: CLOUD-BASED LIBRARY MANAGEMENT SYSTEM	4
Figure 1: AWS Architecture.....	5
Figure 2: AWS Learner Lab Details Panel	8
Figure 3: AWS CLI Credentials Configuration.....	8
Figure 4–6: Cloud9 IDE Workspace.....	9-10
Figure 7–8: Custom VPC Dashboard	11
Figure 9: VPC Resource Map.....	12
Figure 10–11: Subnet Configuration.....	13
Figure 12–14: Internet Gateway Attachment.....	14-15
Figure 15–16: Route Table Configuration.....	15-16
Figure 17: Security Groups Overview	16
Figure 18–19: App / ALB Security Group Rules.....	17
Figure 20: Database Security Group Rules.....	18
Figure 21: RDS Dashboard	18
Figure 22: Database Instance Class & RDS Connectivity Endpoint	19
Figure 23: Subnet Group Configuration.....	20
Figure 24: S3 Bucket Overview	20
Figure 25–27: Uploaded Objects Preview	22
Figure 28: Docker Build Output	23
Figure 29: ECR Repositories List.....	23
Figure 30: ECR Image Details	24
Figure 31–32: Load Balancer Dashboard	24-25
Figure 33: Target Group Configuration & Target Health Check.....	25
Figure 34–35: ECS Cluster Status, Service Deployment & Event Logs	26
Figure 36: Task Definition Blueprint	27
Figure 37–45: Frontend Application Output.....	28-34
Figure 46–48: CloudWatch RDS Performance and CPU Utilization	33

SEMESTER PROJECT: CLOUD-BASED LIBRARY MANAGEMENT SYSTEM

1. PROJECT PROPOSAL, EXECUTIVE SUMMARY & PROBLEM DEFINITION

1.1 Executive Summary

This project outlines the design and deployment of a scalable, cloud-native Library Management System (LMS) on Amazon Web Services (AWS). Traditional local LMS solutions suffer from limited accessibility, poor scalability, and risk of data loss. This solution leverages AWS containerization (ECS & ECR), managed databases (RDS), and secure networking (VPC) to create a robust application. The project successfully demonstrates the migration of a local application to a cloud environment, ensuring high availability and automated infrastructure management within the constraints of the AWS Learner Lab environment.

DNS Link: library-alb-1340845113.us-east-1.elb.amazonaws.com

1.2 Introduction

The Library Management System is a critical tool for educational institutions, managing the circulation of books, member records, and inventory. As institutions grow, on-premise servers fail to handle increased traffic and data loads. This project migrates a modern LMS codebase to AWS, utilizing **Cloud9** for development, **Elastic Container Service (ECS)** for orchestration, and **Relational Database Service (RDS)** for persistent storage.

1.3 Problem Statement

Most libraries rely on legacy, on-premise software that requires expensive hardware maintenance and lacks remote accessibility.

- **Scalability:** Physical servers cannot handle sudden spikes in user traffic during exam periods.
- **Data Integrity:** Local backups are prone to corruption and hardware failure.
- **Accessibility:** Users cannot access the catalog or renew books remotely without complex VPNs.

1.4 Aim & Objectives

Aim: To deploy a fully functional, containerized Library Management System on AWS that ensures high availability and secure data management.

Objectives:

1. **Network Design:** Implement a custom Virtual Private Cloud (VPC) with public/private subnets and internet gateways.
2. **Containerization:** Dockerize the LMS application and store images securely in Amazon Elastic Container Registry (ECR).
3. **Object Storage:** Implement Amazon S3 to offload static assets (User Profiles, Book Covers) from the container, ensuring stateless application design.
4. **Orchestration:** Deploy the application using Amazon ECS with Target Groups for traffic distribution.
5. **Database Management:** Utilize Amazon RDS for a managed, reliable SQL database backend.
6. **Security:** Implement network security via Security Groups and utilize the restricted LabRole for service authorization.
7. **Observability:** Configure Amazon CloudWatch to monitor container health, CPU utilization, and application logs in real-time.

2. AWS ARCHITECTURE DESIGN & IMPLEMENTATION

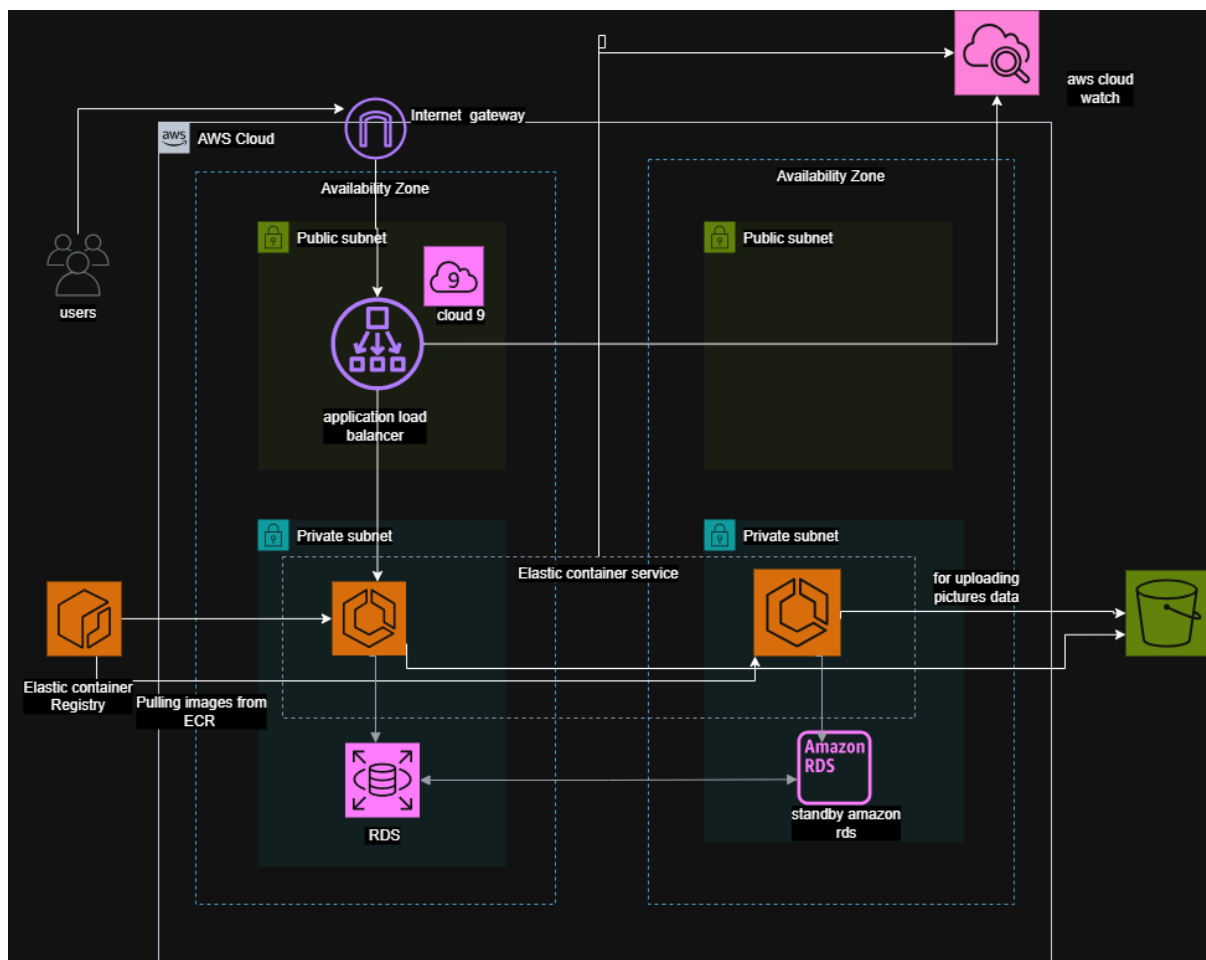


Figure 1: AWS Architecture

2.1 AWS Architecture Description

The architecture follows a **multi-tier microservices pattern**:

1. **Network Layer:** A **VPC** hosting two subnets. A **Public Subnet** contains the **Internet Gateway** and **Application Load Balancer (ALB)** / Target Groups.
2. **Compute Layer:** **Amazon ECS** (Elastic Container Service) runs the LMS Docker containers. **Cloud9** is used as the development environment (IDE) to build and push images.
3. **Storage Layer:**
 - **Relational:** Amazon RDS (MySQL) resides in the private network scope to store relational data (book/user records).
 - **Object:** **Amazon S3** is used to store unstructured data such as book cover images and user profile pictures, reducing the storage burden on the application container.
4. **Security Layer:** Security groups act as virtual firewalls. IAM access is managed via the pre-defined LabRole.
5. **Monitoring Layer:** Amazon CloudWatch aggregates logs from ECS tasks and tracks performance metrics (CPU/Memory usage) to ensure system reliability.

2.2 Implementation Steps

Phase 1: Network Setup (VPC)

1. **VPC Creation:** Created a custom VPC (MERN-Project-VPC) in the us-east-1 region.
2. **Subnets & Route Tables:**
 - Created Public Subnets for internet access.
 - Created an **Internet Gateway (IGW)** and attached it to the VPC to allow outbound traffic.
 - Configured **Route Tables** to direct traffic 0.0.0.0/0 to the IGW.

Phase 2: Database Setup (RDS)

1. **Service Selection:** Navigated to RDS console.
2. **Configuration:** Launched a db.t3.micro instance (Burstable class) to stay within Learner Lab budget limits.
3. **Security:** Configured the security group to allow inbound traffic on port 3306 (MySQL) or 5432 (PostgreSQL) only from the ECS tasks/Security Group.
 - *Constraint Note:* Enhanced monitoring was disabled as per lab restrictions.

Phase 3: Object Storage Setup (S3)

1. **Bucket Creation:** Created a unique S3 bucket (e.g., lms-static-assets) in the us-east-1 region.
2. **Permissions:**
 - Disabled "Block All Public Access" (or configured specific ACLs) to allow the application to retrieve images.

- Configured Cross-Origin Resource Sharing (CORS) to allow the web frontend to fetch images directly from the bucket.
- 3. **Integration:** Updated the app.py configuration to include AWS_S3_BUCKET_NAME and S3_REGION environment variables.

Phase 4: Development & Container Registry (Cloud9 & ECR)

1. **Environment:** Launched an AWS Cloud9 environment to access the CLI and git.
2. **Source Control:** Cloned the repository: `git clone https://github.com/BSSE-23050/LMS.git`.
3. **Docker Build:** Used the terminal to build the docker image.
 - *Command:* `docker build -t lms-app .`
4. **ECR Repository:** Created a private repository in Amazon ECR.
5. **Push to ECR:** Authenticated Docker with AWS CLI and pushed the image to the repository.

Phase 5: Deployment (ECS & Target Groups)

1. **Target Group:** Created a Target Group (library-tg2) to route HTTP traffic (Port 80) to the container port.
2. **Task Definition:** Created a Task Definition using the **ECR Image URI**.
 - *IAM Role Configuration:* Assigned the **LabRole** as the "Task Role" and "Task Execution Role" because the Learner Lab restricts creating new IAM roles.
3. **Service Creation:** Deployed an ECS Service with the defined task, mapped to the Target Group.
4. **Log Configuration:** configured the Task Definition to use the awslogs driver, automatically streaming all container stdout/stderr output to Amazon CloudWatch Logs.

2.3 Security & IAM

Challenge: The AWS Learner Lab environment has strict restrictions on IAM. Users "cannot create roles, except... service-linked roles". **Solution:**

- **Role Usage:** We utilized the pre-existing **LabRole**. This role is designed to be attached to EC2, Lambda, or ECS tasks and grants necessary permissions for services to interact.
 - **Credentials:** For CLI access within the lab (or local development), we utilized the temporary AWS Access Keys (`aws_access_key_id`, `aws_secret_access_key`, and `aws_session_token`) provided by the "AWS Details" panel in the lab interface. These tokens change every session.
 - **Network Security:** Security Groups were configured to ensure the RDS database is not publicly accessible, only accepting connections from the application layer.
-

3. SCREENSHOTS OF THE IMPLEMENTATION

PHASE 1: LAB ENVIRONMENT & AUTHENTICATION

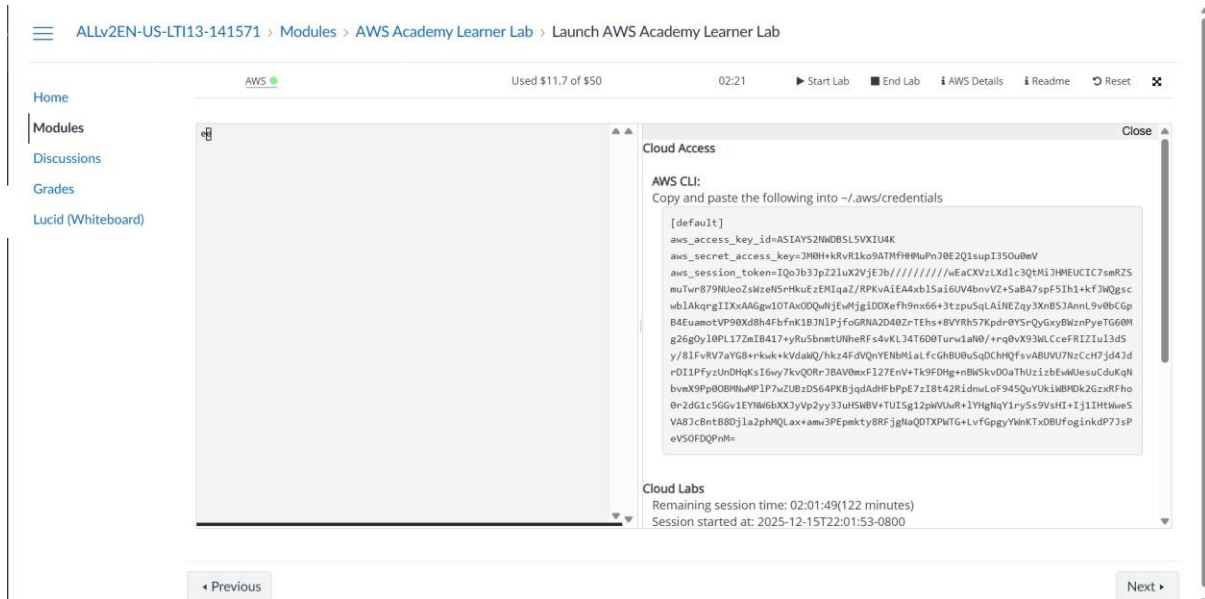


Figure 2: AWS Learner Lab Details Panel

- **Description:** The starting point of the lab environment. This panel provides the temporary AWS Access Key, Secret Key, and Session Token required for programmatic access to AWS services.

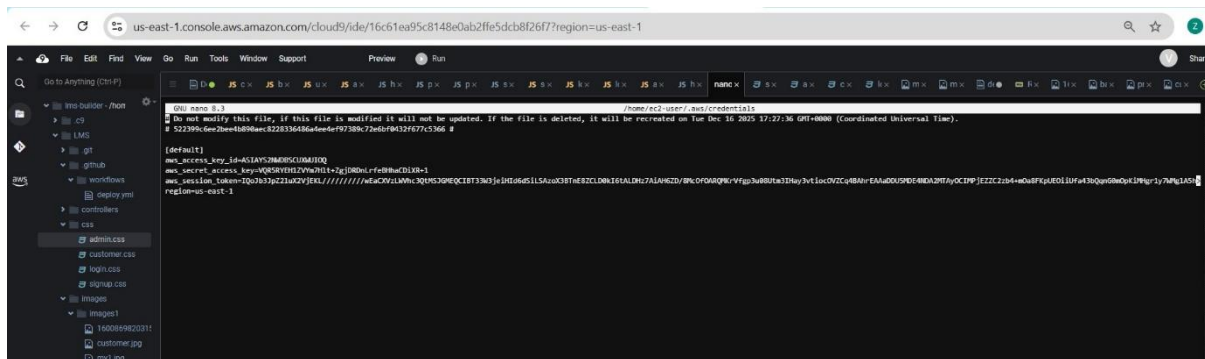


Figure 3: AWS CLI Credentials Configuration

- **Description:** The ~/.aws/credentials file inside the Cloud9 environment. This screenshot demonstrates the manual injection of session tokens to authenticate the AWS CLI, bypassing the restriction on creating new IAM execution roles.

Semester x SEMESTER x Launch All x Task defini x AWS Cloud x lms-build x lms-build x Books x Google G x New Tab x

us-east-1.console.aws.amazon.com/cloud9/ide/16c61ea95c8148e0ab2ffe5dcb8f26f7?region=us-east-1

File Edit Find View Go Run Tools Window Support Preview Run Share

Go to Anything (Ctrl-P)

lms-builder ./.ho

cs

LMS

git

github

aws

workflows

deploy.yml

controllers

css

admin.css

customer.css

login.css

signup.css

images

images1

160086982031

customer.jpg

my1.jpg

my2.jpg

background.jpg

profile-pic.jpg

models

node_modules

validation_rules

views

gitignore

JS app.js

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package.json ./
4 RUN npm install --production
5 COPY . .
6 # EXPOSE: Make sure this says 8080, not 3000!
7 EXPOSE 8080
8 CMD ["node", "app.js"]
```

8:23 Dockerfile Spaces: 4

Replace With Replace Replace All

Semester x SEMESTER x Launch All x Task defini x AWS Cloud x lms-build x lms-build x Books x Google G x New Tab x

us-east-1.console.aws.amazon.com/cloud9/ide/16c61ea95c8148e0ab2ffe5dcb8f26f7?region=us-east-1

File Edit Find View Go Run Tools Window Support Preview Run Share

Go to Anything (Ctrl-P)

workflows

deploy.yml

controllers

css

admin.css

customer.css

login.css

signup.css

images

images1

160086982031

customer.jpg

my1.jpg

my2.jpg

background.jpg

profile-pic.jpg

models

node_modules

validation_rules

views

gitignore

JS app.js

Dockerfile

library_manager

package-lock.json

package.json

README.md

```
1 //declaration
2 var express = require('express');
3 var expressSession = require('express-session');
4 var bodyParser = require('body-parser');
5 var app = express();
6 var port = 8080;
7
8 //controllers
9 var signup = require('./controllers/signup');
10 var login = require('./controllers/login');
11 var logout = require('./controllers/logout');
12
13 //admin controllers
14 var admin = require('./controllers/admin');
15
16
17 //customer controllers
18 var customer = require('./controllers/customer');
19
20 //configure
21 app.set('view engine', 'ejs');
22
23 //middlewares
24 app.use(bodyParser.urlencoded({extended: false}));
25 app.use(expressSession({secret: 'my too secret pass', resave: false, saveUninitialized: true}));
26 app.use('/css', express.static(_dirname + '/css'));
27 app.use('/images', express.static(_dirname + '/images'));
28
29 app.use('*', function(req, res, next){
30   if(req.originalUrl == '/login' || req.originalUrl == '/signup')
31   {
32     next();
33   }
34   else
35   {
36     if(!req.session.admin || req.session.customer)
37     {
38       res.redirect('/login');
39       return;
40     }
41     next();
42   }
43 }
```

32:3 JavaScript Tabs: 4

Replace With Replace Replace All

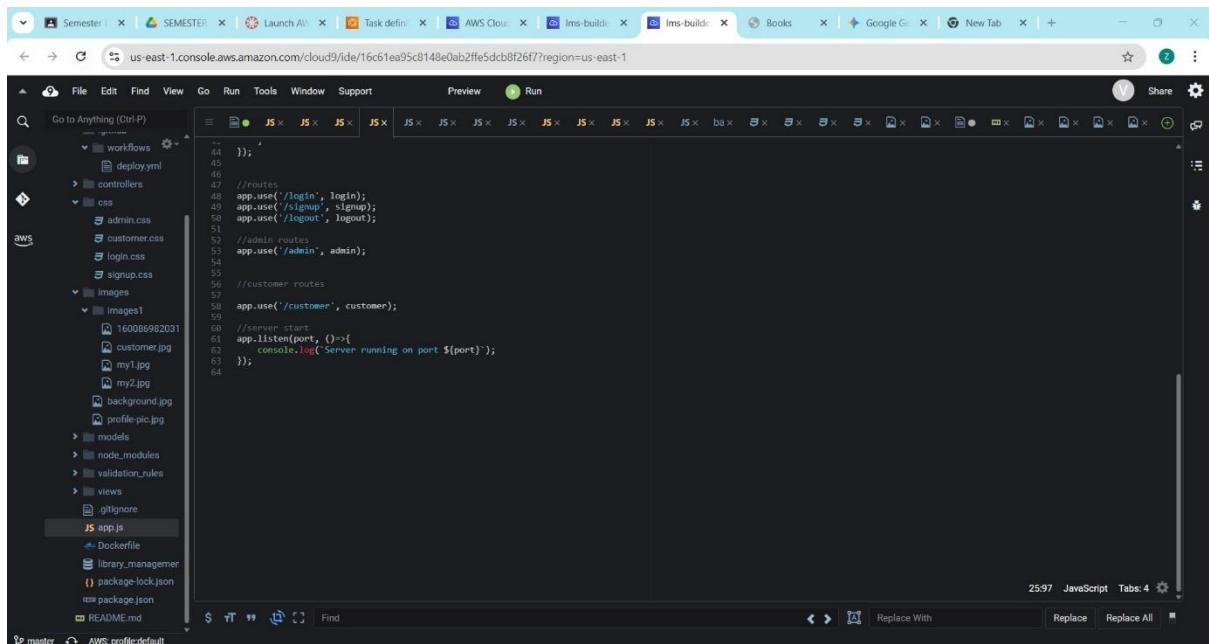


Figure 4, 5, 6: Cloud9 IDE Workspace

- **Description:** The active AWS Cloud9 development environment. The file tree displays the project structure, including app.py, and the Dockerfile, confirming the code is present in the cloud environment.

PHASE 2: NETWORK INFRASTRUCTURE (VPC)

VPC dashboard

Introducing VPC encryption control
Manage and enforce encryption settings across your Virtual Private Cloud (VPC) resources. This centralized control helps ensure compliance with security policies and data protection regulations.

Your VPCs

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 C
MERN-Project-VPC	vpc-037a3cc42d5eb1200	Available	-	-	Off	10.0.0.0/16
-	vpc-0f6f1f15b6f1fed57	Available	-	-	Off	172.31.0.0/16

vpc-037a3cc42d5eb1200 / MERN-Project-VPC

Details

VPC ID vpc-037a3cc42d5eb1200	State Available	Block Public Access Off	DNS hostnames Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-031fd46f244bfc62	Main route table rtb-0bc77c44ec5b51368
Main network ACL acl-0e1b1978e25d6f427	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group)	Network Address Translation	Route 53 Resolver DNS Firewall rule	Owner ID

Figure 7, 8: Custom VPC Dashboard

- Description:** The VPC console listing the custom MERN-Project-VPC. It shows the allocated IPv4 CIDR block (10.0.0.0/16), confirming the creation of an isolated network space for the project.

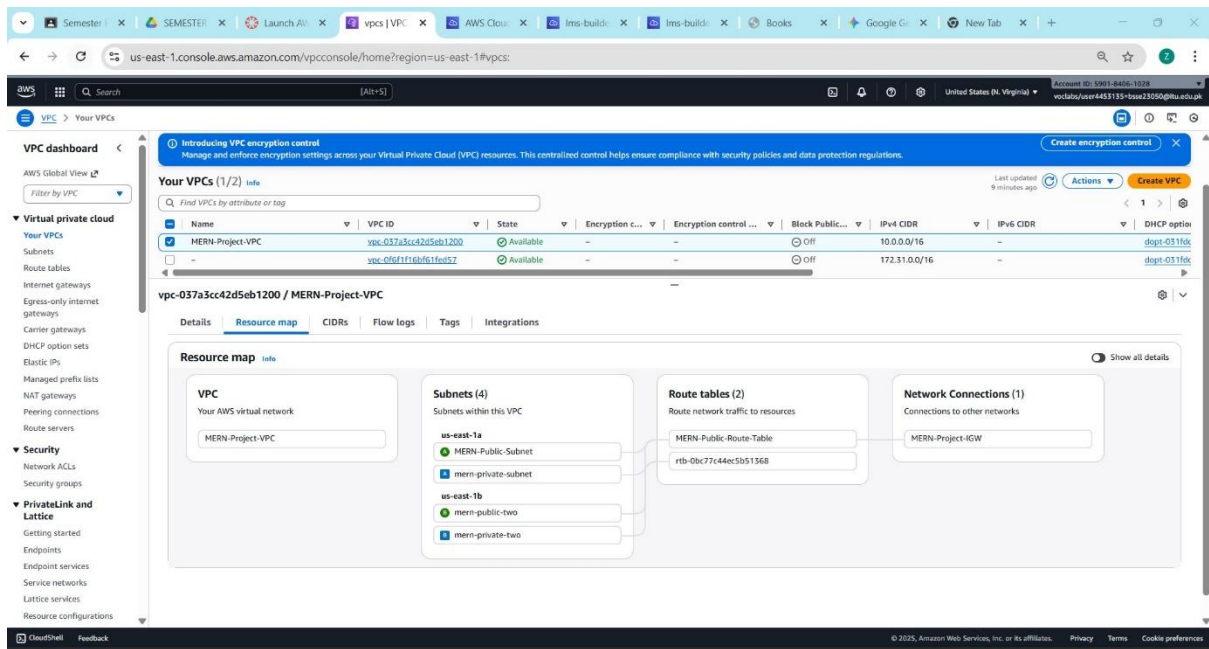


Figure 9: VPC Resource Map

- **Description:** A topology diagram generated by AWS. It visually maps the connections between the MERN-Project-VPC, its Subnets, Route Tables, and the Internet Gateway, providing a high-level architectural view.

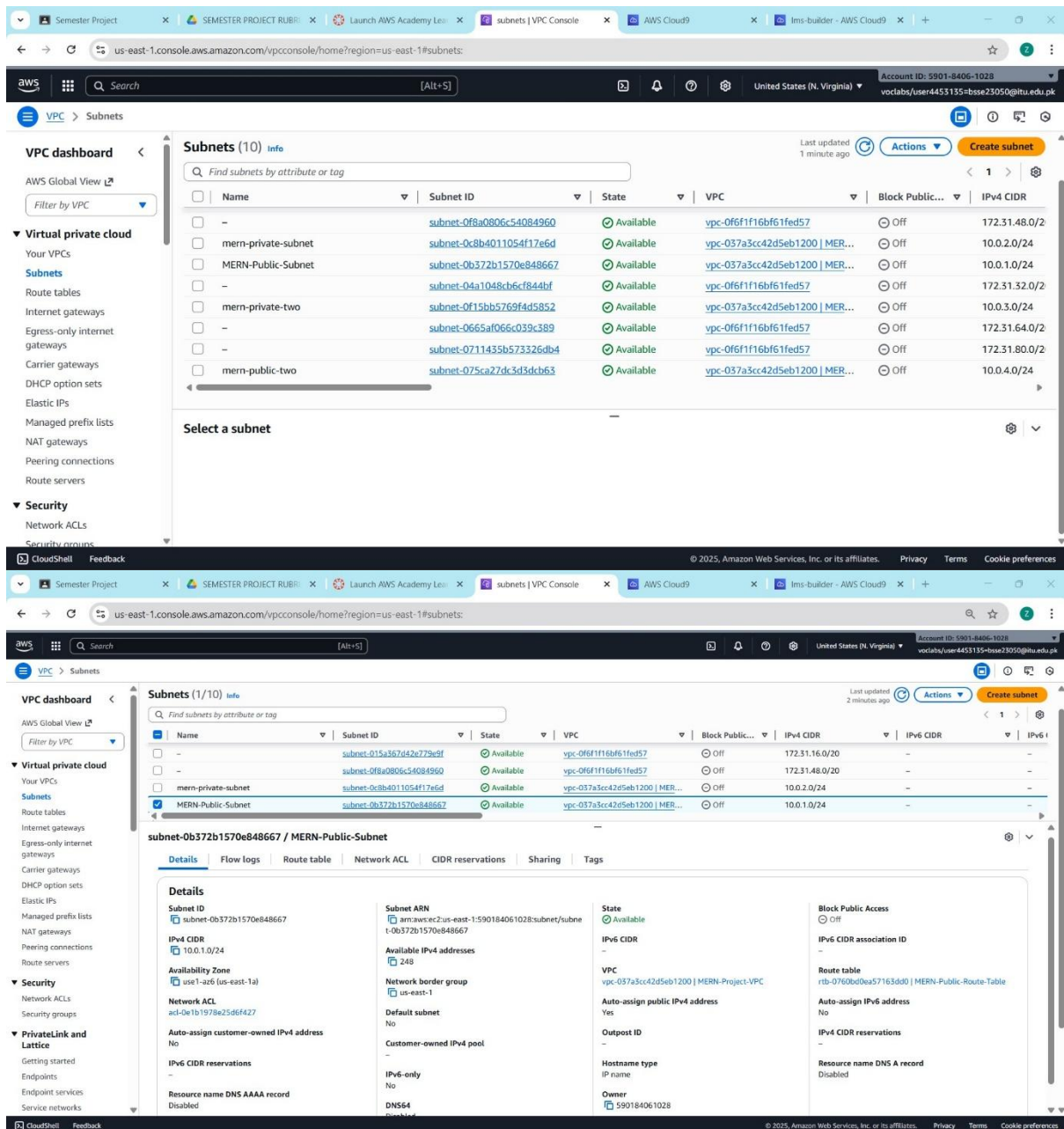


Figure 10, 11: Subnet Configuration

- Description:** The list of subnets created within the VPC. It distinguishes between the **Public Subnet** (for the Load Balancer) and the **Private Subnet** (for the Database), adhering to security best practices.

Semester Project x SEMESTER PROJECT RUB x Launch AWS Academy Le x RouteTables | VPC Console x AWS Cloud9 x lms-builder - AWS Cloud9 x +

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTables:

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTables:

VPC > Route tables

VPC dashboard <

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

Route tables (1/3) info

Find route tables by attribute or tag

Last updated 3 minutes ago

Actions Create route table

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
-	rtb-0863cc197277a29d	-	-	Yes	vpc-0f6f1f162b61fed57	590184061028
MERN-Public-Route-Table	rtb-0760bd0ea57163dd0	2 subnets	-	No	vpc-037a3cc42d5eb1200 MER...	590184061028
-	rtb-0bc77c44ec5b51368	-	-	Yes	vpc-037a3cc42d5eb1200 MER...	590184061028

rtb-0760bd0ea57163dd0 / MERN-Public-Route-Table

Details Routes Subnet associations Edge associations Route propagation Tags

Details

Route table ID rtb-0760bd0ea57163dd0

Main No

Explicit subnet associations 2 subnets

Edge associations -

VPC vpc-037a3cc42d5eb1200 | MERN-Project-VPC

Owner ID 590184061028

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Semester Project x SEMESTER PROJECT RUB x Launch AWS Academy Le x RouteTables | VPC Console x AWS Cloud9 x lms-builder - AWS Cloud9 x +

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTables:

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTables:

VPC > Route tables

VPC dashboard <

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

Route tables (1/3) info

Find route tables by attribute or tag

Last updated 3 minutes ago

Actions Create route table

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
-	rtb-0863cc197277a29d	-	-	Yes	vpc-0f6f1f162b61fed57	590184061028
MERN-Public-Route-Table	rtb-0760bd0ea57163dd0	2 subnets	-	No	vpc-037a3cc42d5eb1200 MER...	590184061028
-	rtb-0bc77c44ec5b51368	-	-	Yes	vpc-037a3cc42d5eb1200 MER...	590184061028

rtb-0760bd0ea57163dd0 / MERN-Public-Route-Table

Details Routes Subnet associations Edge associations Route propagation Tags

Routes (2)

Filter routes

Both Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	lgw-0b88758cc0307c5e4	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

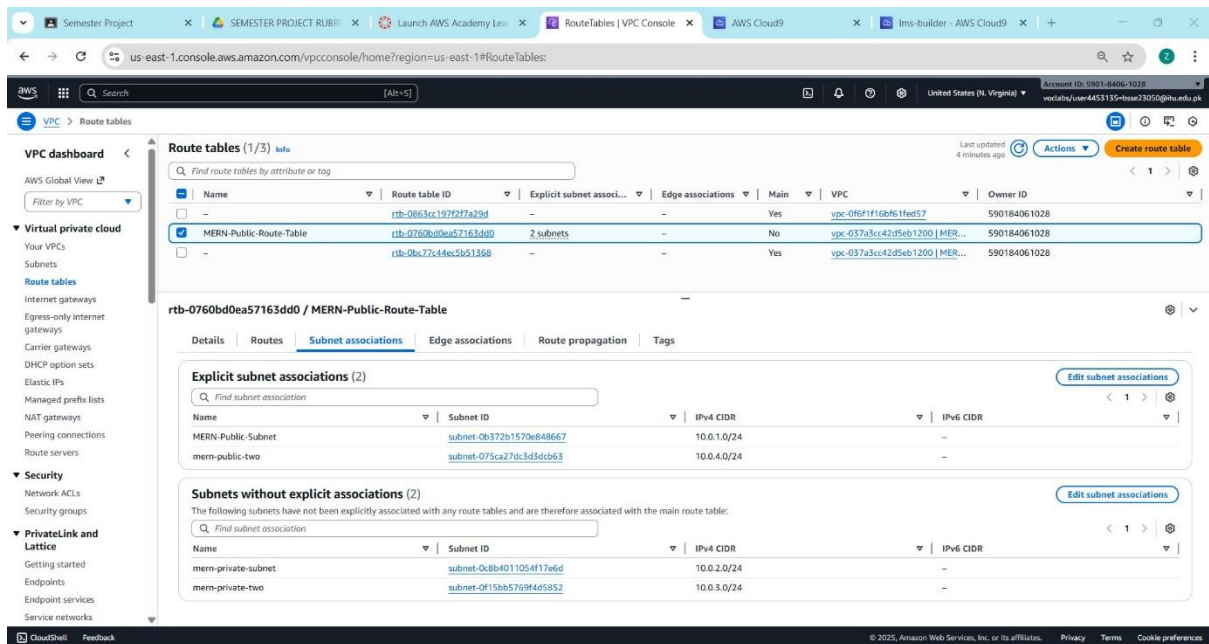
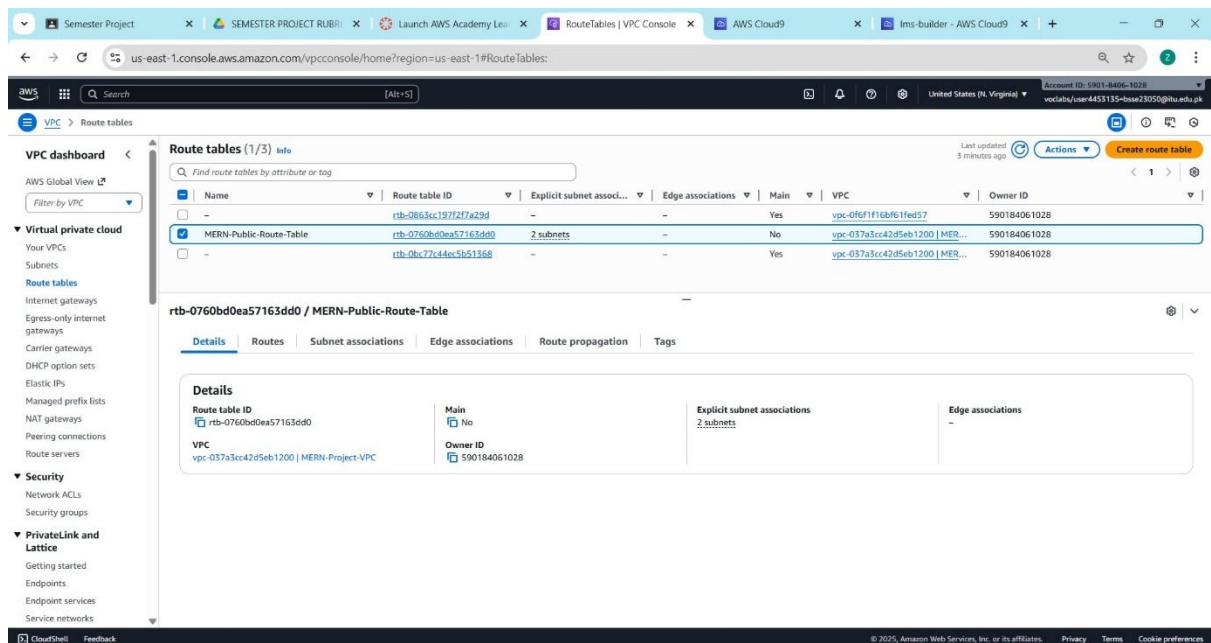


Figure 12, 13, 14: Internet Gateway Attachment

- Description:** The Internet Gateway (IGW) attached to the VPC. This component is essential for allowing the application containers in the public subnet to communicate with the external internet.



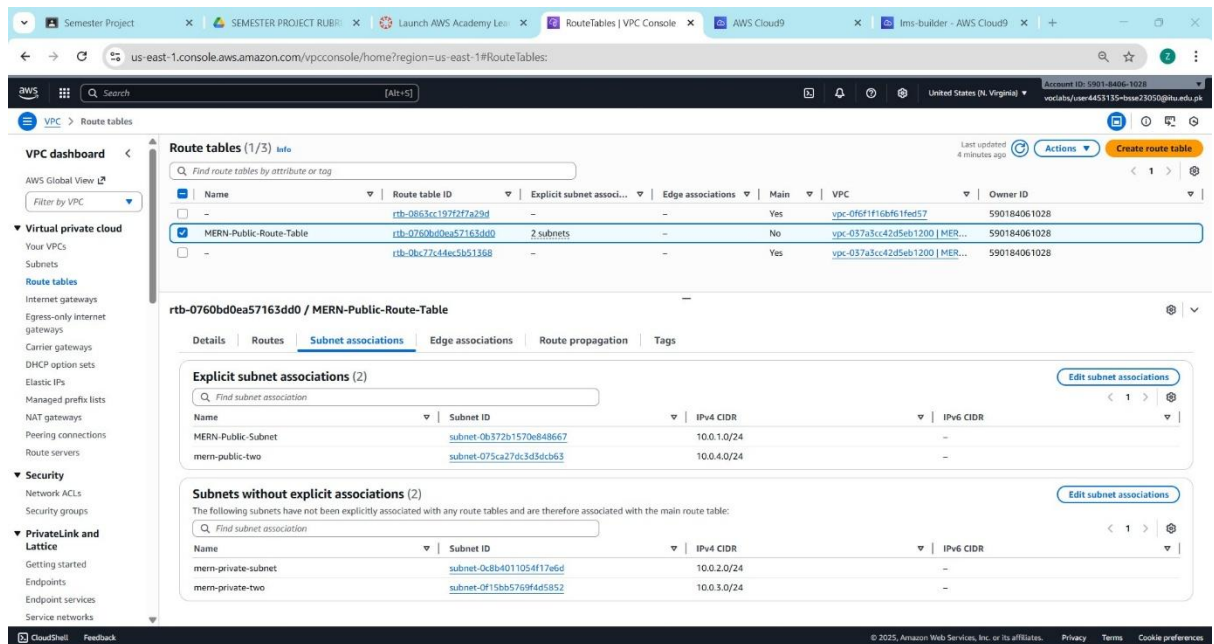


Figure 15, 16: Route Table Configuration

- **Description:** The Route Table associated with the public subnet. The entry 0.0.0.0/0 -> igw-xxxx confirms that internet-bound traffic is correctly routed through the gateway.

PHASE 3: SECURITY & ACCESS CONTROL

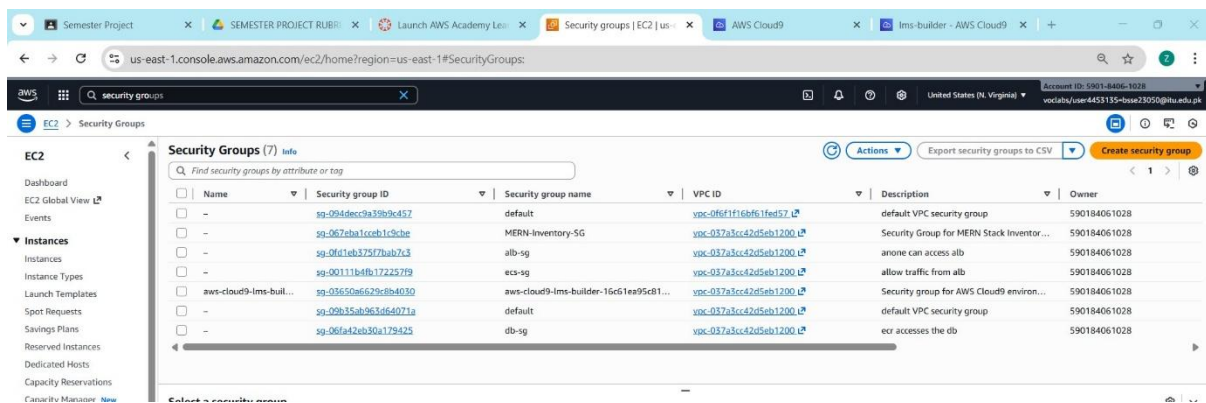


Figure 17: Security Groups Overview

- **Description:** The list of Security Groups created for the project. It shows distinct groups for the Load Balancer, the Application (ECS), and the Database (RDS), ensuring granular control over traffic.

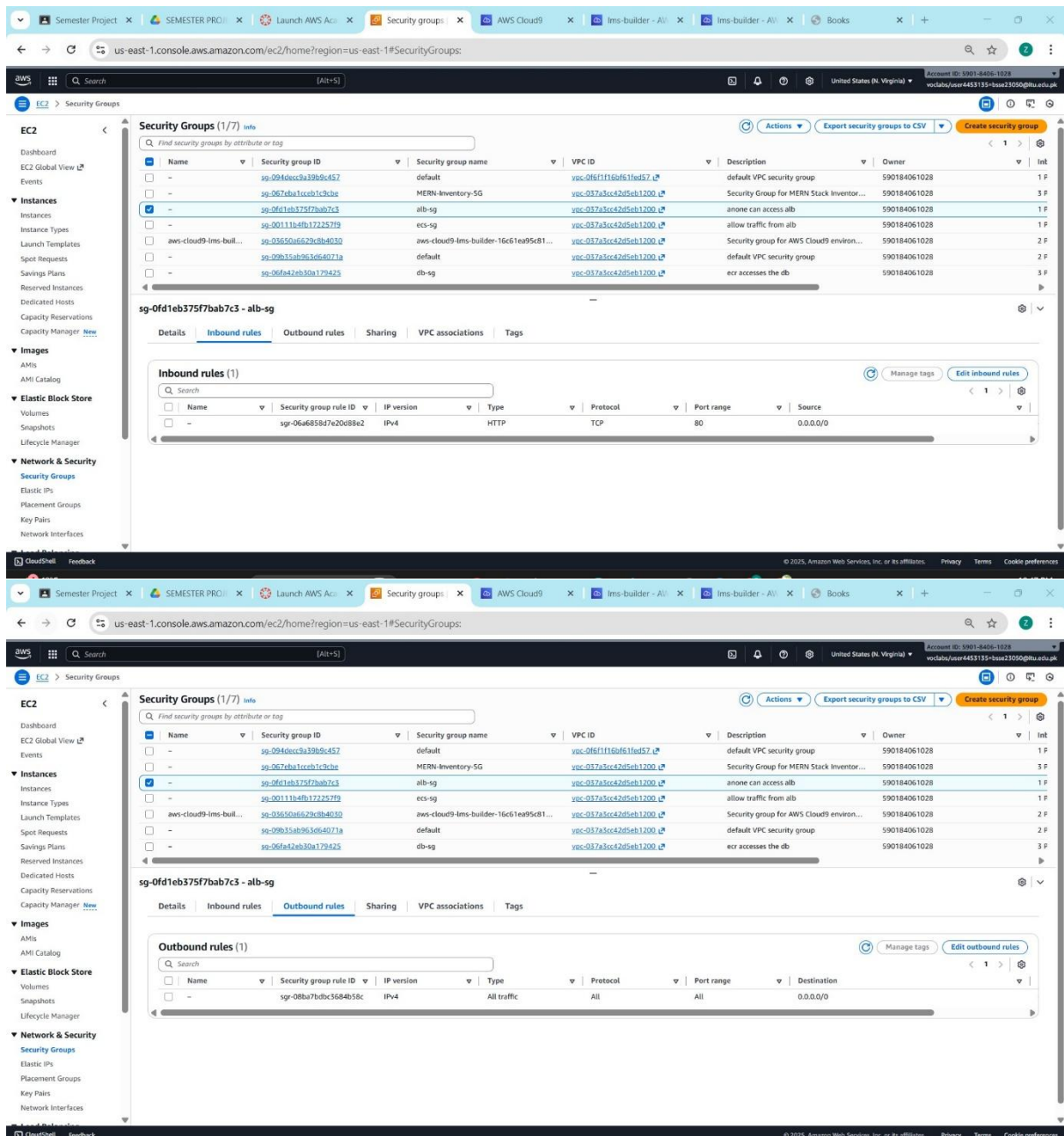


Figure 18, 19: App/ALB Security Group Rules

- Description:** The Inbound Rules for the Application Security Group. It shows that HTTP traffic (Port 80) is allowed from the internet, enabling user access to the website.

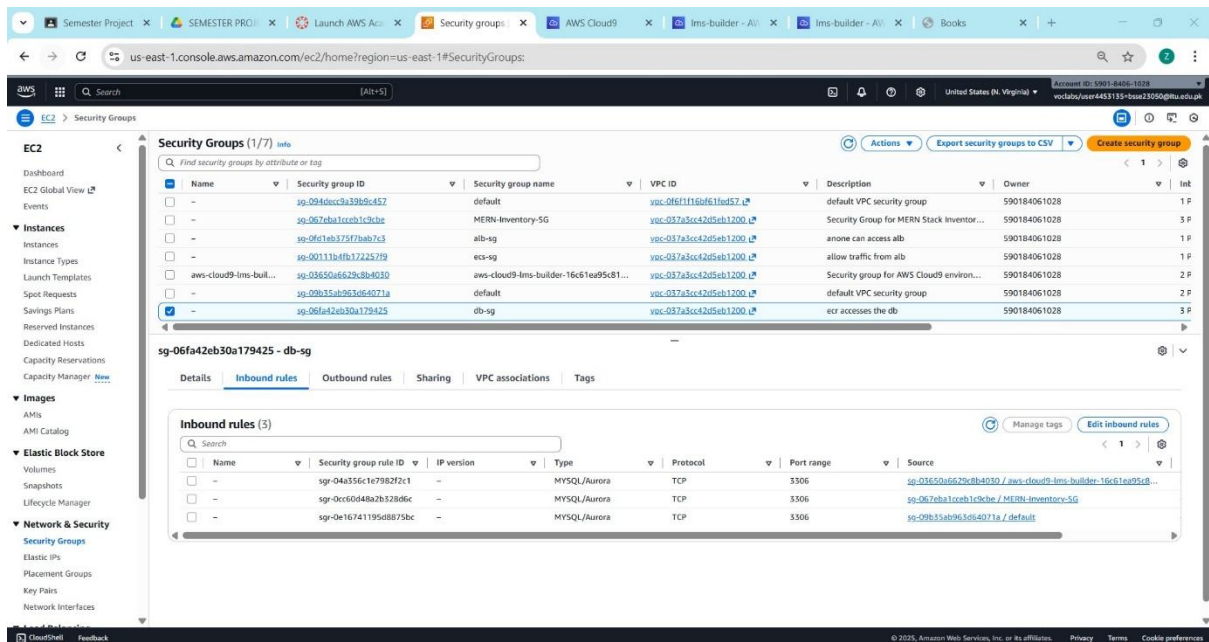


Figure 20: Database Security Group Rules

- Description:** The restricted Inbound Rules for the RDS Security Group. It demonstrates that traffic on Port 3306 (MySQL) is **only** allowed from the Application Security Group ID, preventing public access to the data.

PHASE 4: DATABASE DEPLOYMENT (RDS)

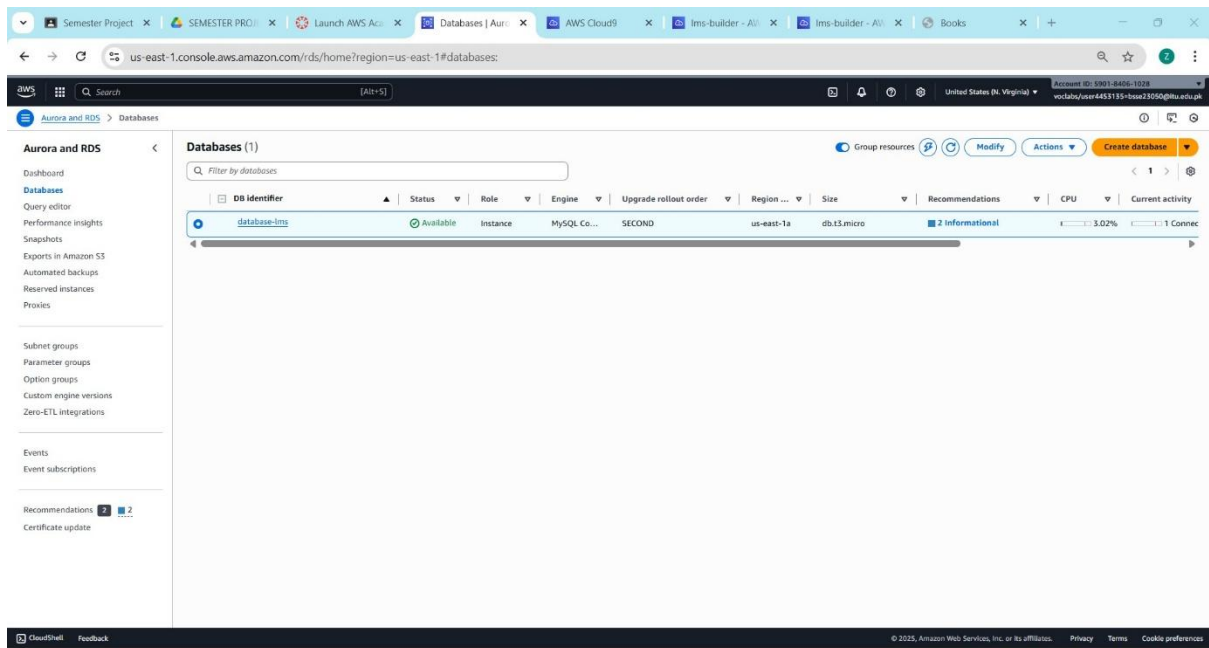


Figure 21: RDS Dashboard

- **Description:** The Amazon RDS console showing the database-lms instance. The status is "Available", confirming the database is successfully provisioned.

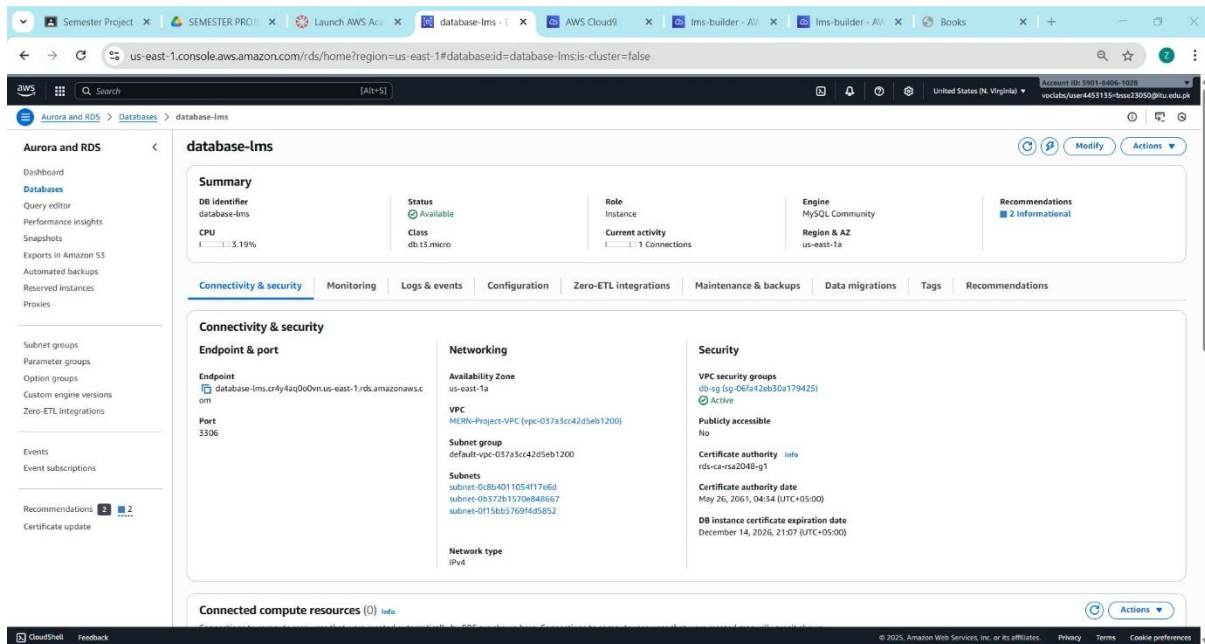


Figure 22: Database Instance Class & RDS Connectivity Endpoint

- **Description:** The configuration details of the RDS instance. It highlights the use of the db.t3.micro class and "Free Tier" settings to optimize costs within the Learner Lab budget.
- **Description:** The specific Endpoint URL and Port number provided by AWS. This string is used by the application code to establish the connection to the managed database.

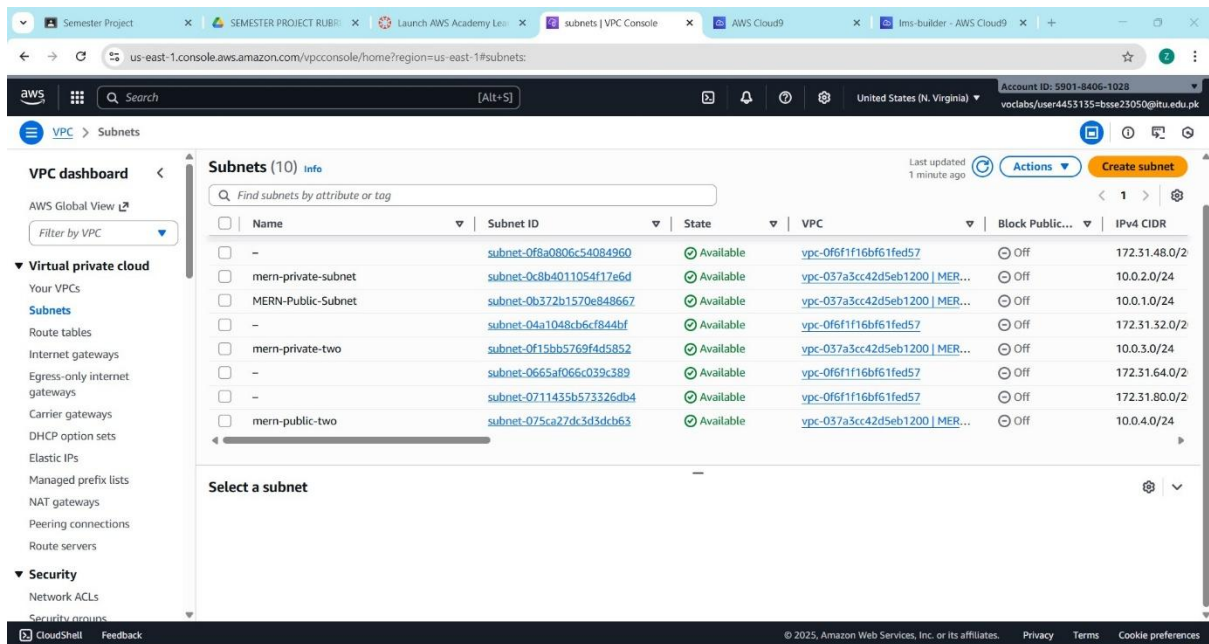


Figure 23: Subnet Group Configuration

- **Description:** The DB Subnet Group details. It confirms that the database is deployed across multiple Availability Zones (AZs) within the private subnets for higher reliability.

PHASE 5: OBJECT STORAGE

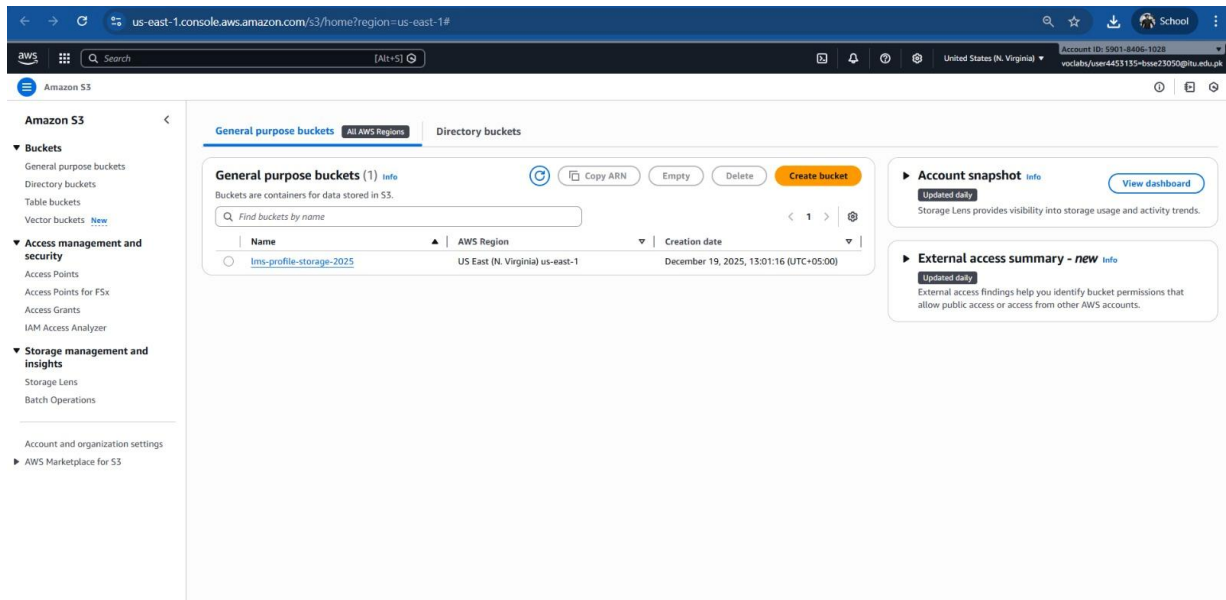


Figure 24: S3 Bucket Overview

- **Description:** The Amazon S3 console displaying the created bucket lms-assets-bucket. This bucket acts as the centralized repository for all static media files (profile pictures and admin dps), decoupling storage from the application compute layer.

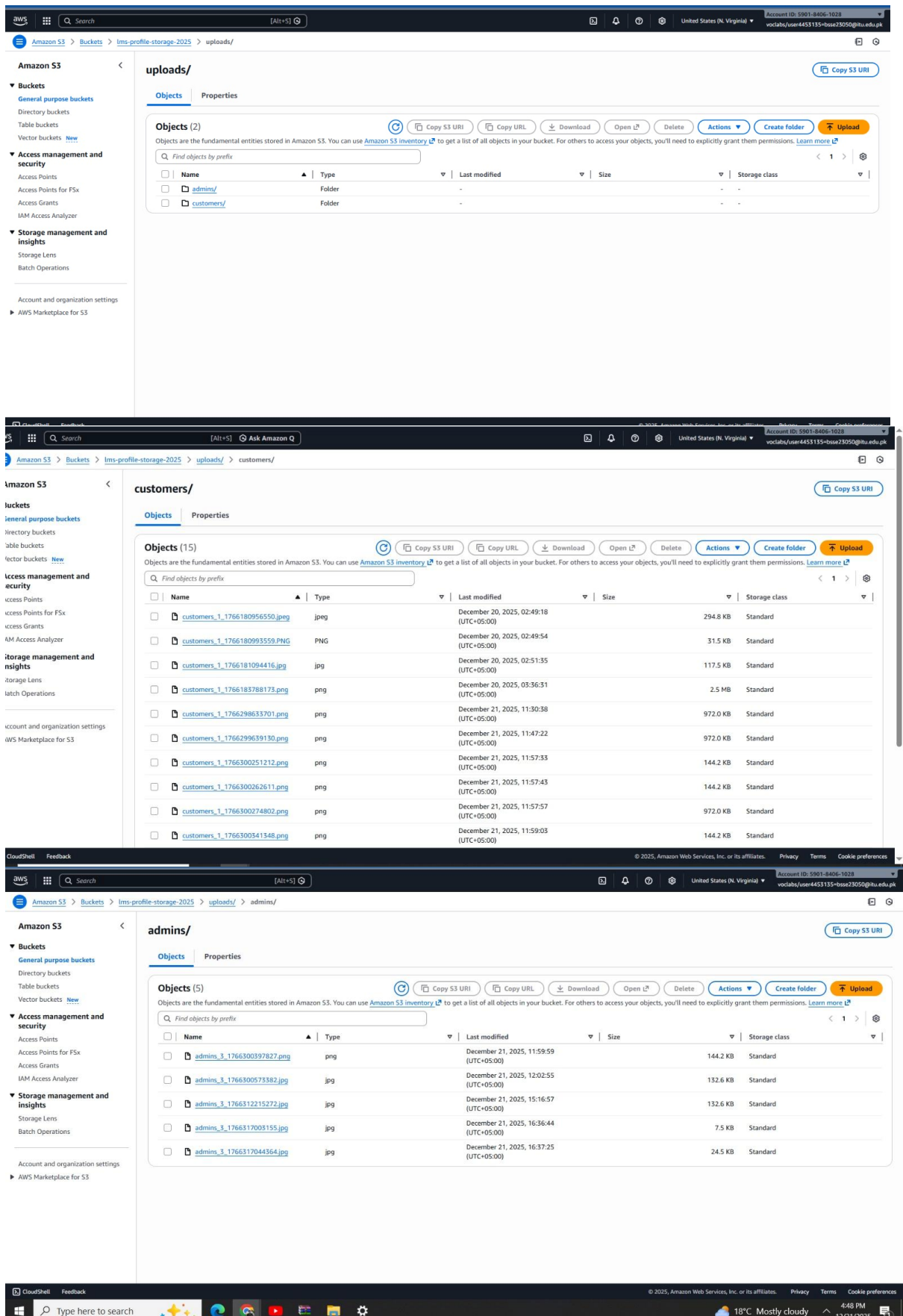


Figure 25,26,27: Uploaded Objects Preview

- ## PHASE 6: CONTAINER REGISTRY (ECR)



-
- The screenshot shows the AWS Management Console for the Amazon Elastic Container Registry (ECR) Private Registry. The browser address bar shows the URL: `us-east-1.console.aws.amazon.com/ecr/private-registry/repositories?region=us-east-1`. The console header includes the AWS logo, a search bar, and the region `United States (N. Virginia)`. The left sidebar contains navigation links for **Private registry**, **Public registry**, **ECR public gallery**, **Amazon ECS**, **Amazon EKS**, **Getting started**, and **Documentation**. The main content area features a blue banner for **Managed signing now available**. Below the banner, the **Private repositories (1)** section includes a search bar and a table of repositories.
- | | Repository name | URI | Created at | Tag immutability | Encryption type |
|-----------------------|-----------------|--|--------------------------------------|------------------|-----------------|
| ☐ | library-app | 590184061028.dkr.ecr.us-east-1.amazonaws.com/library-app | December 14, 2025, 23:48:36 (UTC+05) | Mutable | AES-256 |

Figure 29: ECR Repositories List

- **Description:** The Amazon ECR console showing the creation of the private repository lms-project.

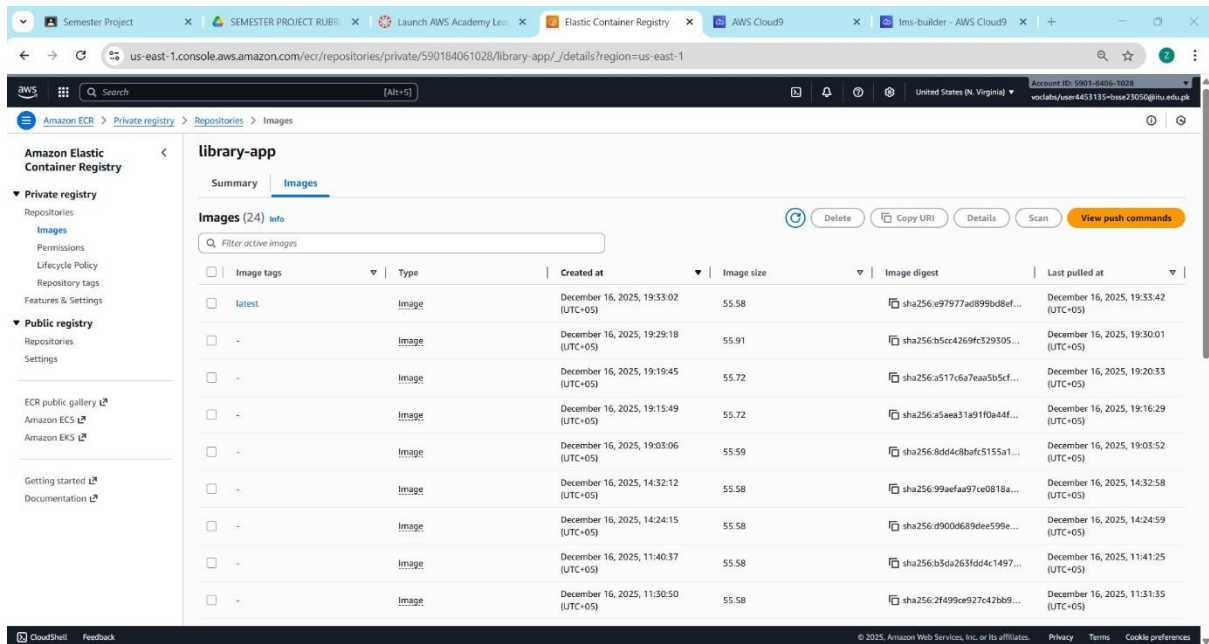
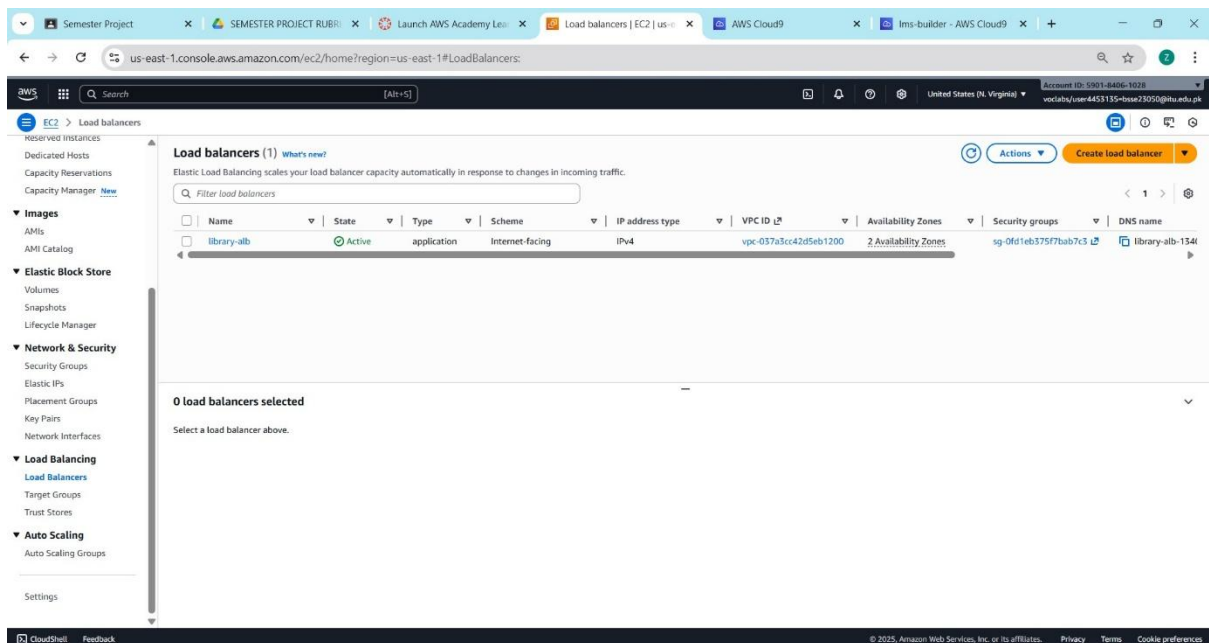


Figure 30: ECR Image Details

- **Description:** The repository contents showing the pushed image. The tag "latest" and the "Image URI" confirm that the Docker image is stored securely and ready for deployment.

PHASE 7: LOAD BALANCING (ALB)



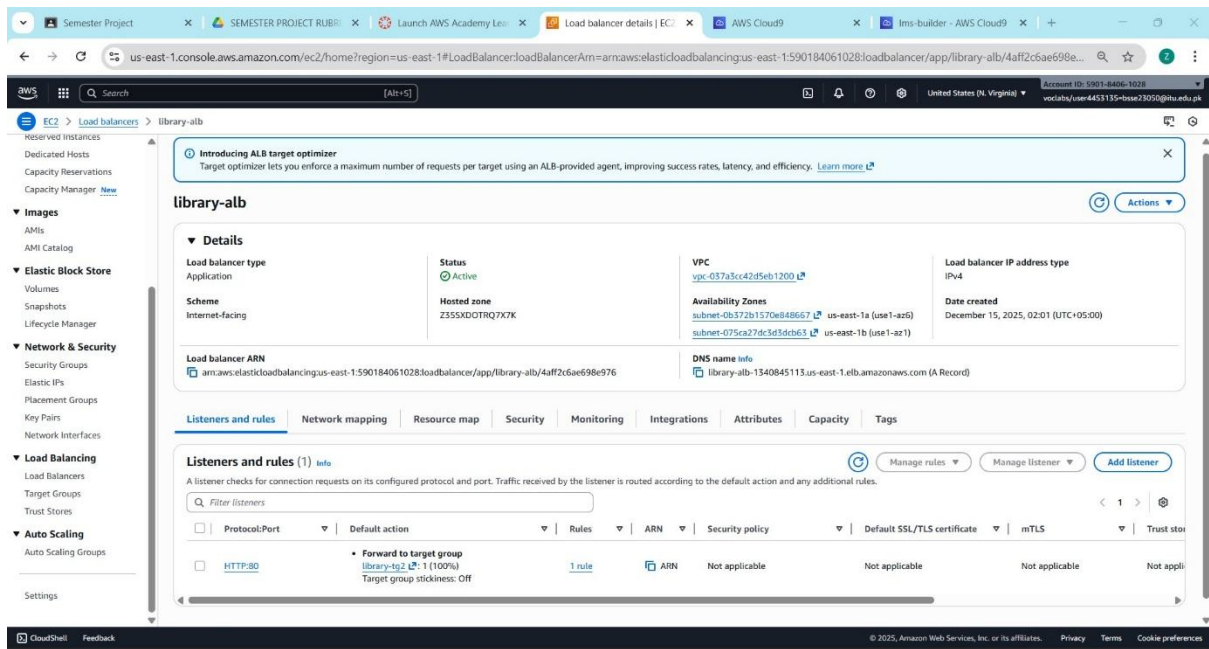


Figure 31, 32: Load Balancer Dashboard

- **Description:** The EC2 Load Balancer console showing the Application Load Balancer (ALB) in the "Active" state. It displays the DNS name used to access the website.

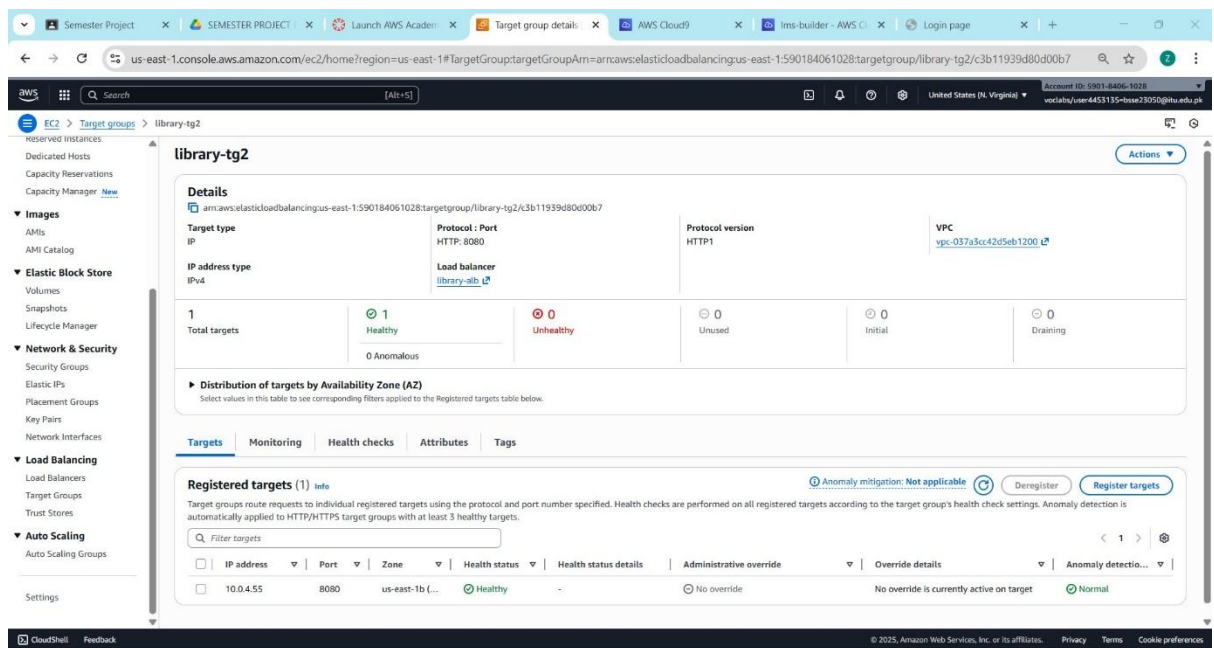


Figure 33: Target Group Configuration & Target Health Check

- **Description:** The Target Group settings. It shows the protocol is HTTP on Port 80 and the target type is "IP", which is required for AWS Fargate tasks.

- **Description:** The "Targets" tab within the Target Group. The status "**Healthy**" confirms the ALB can successfully reach the ECS container and receive a valid response.

PHASE 8: ORCHESTRATION (ECS)

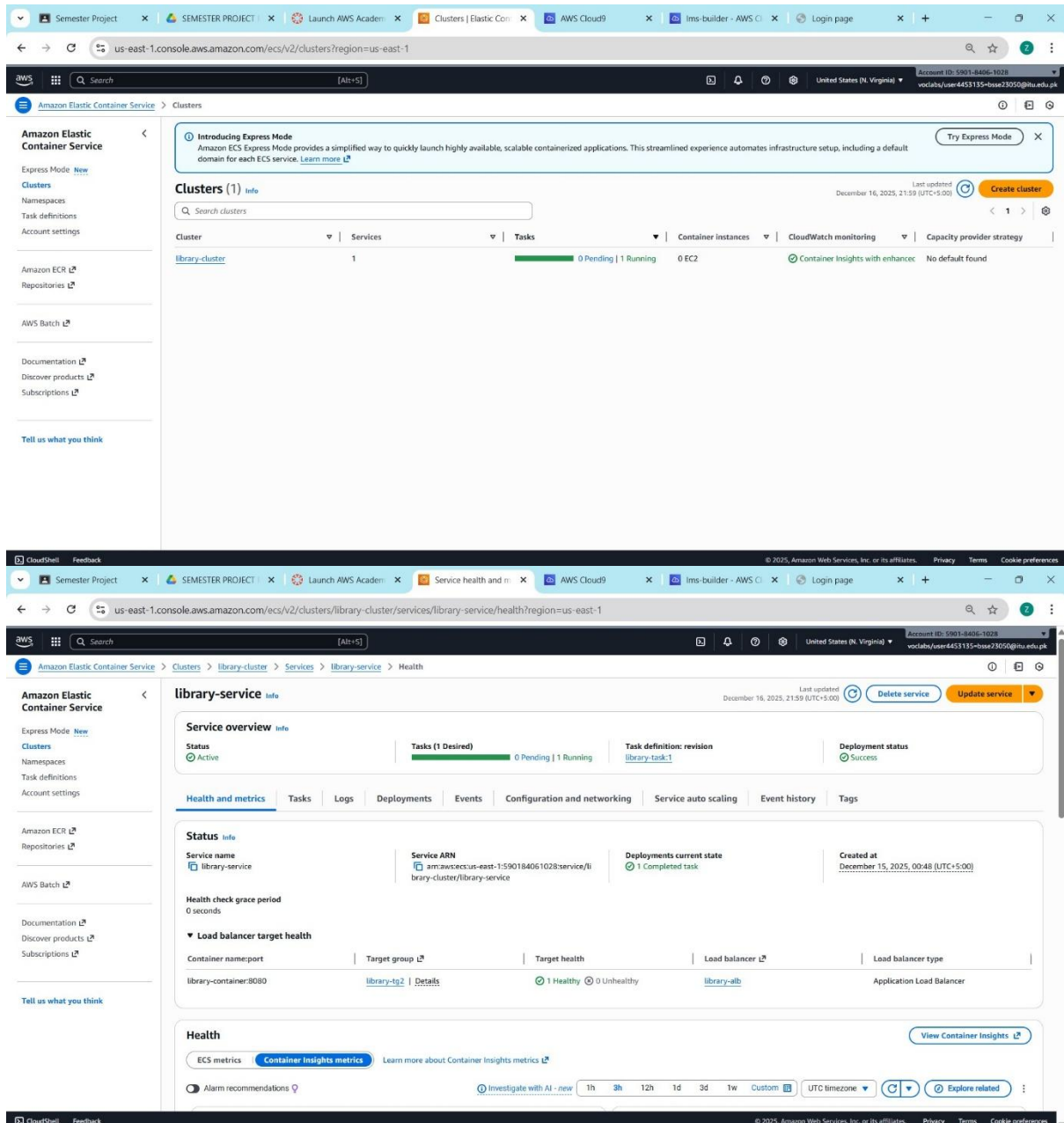


Figure 34, 35: ECS Cluster Status & ECS Service Deployment & Service Event Log

- **Description:** The ECS Cluster dashboard showing the status "**Active**". This cluster serves as the management boundary for the application services.
- **Description:** The ECS Service status showing "**Running tasks: 1**". This confirms the orchestrator has successfully launched the container based on the task definition.

- **Description:** The "Events" tab in the ECS Service. The log "service has reached a steady state" serves as proof of a stable and successful deployment.

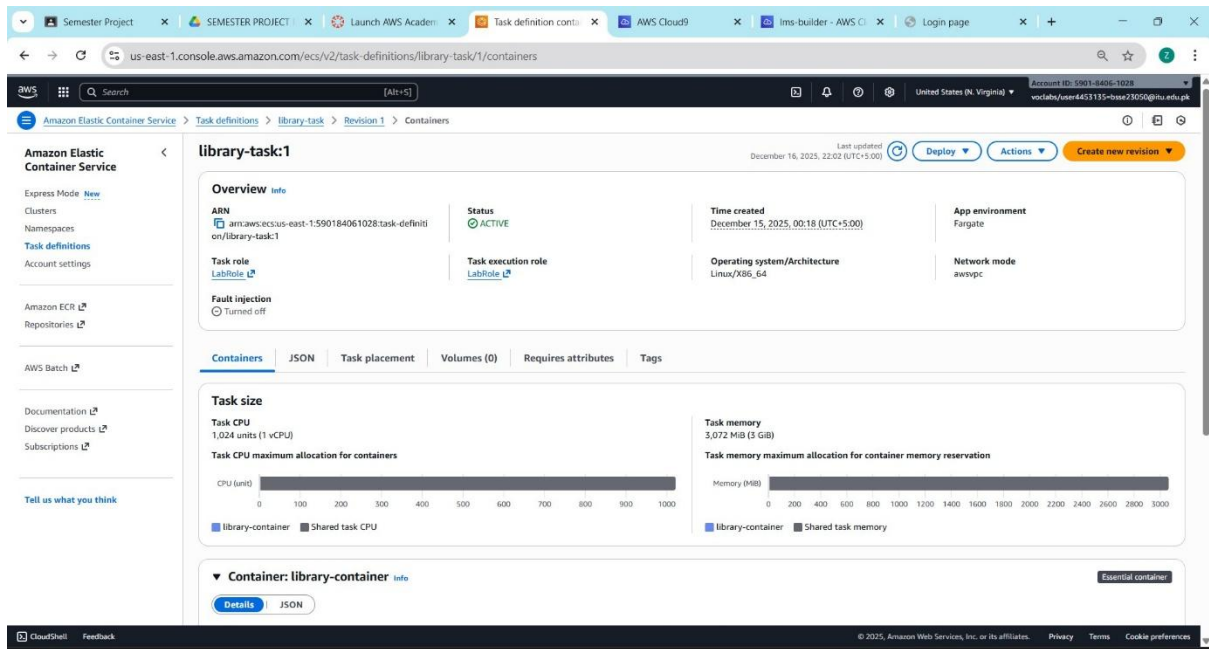
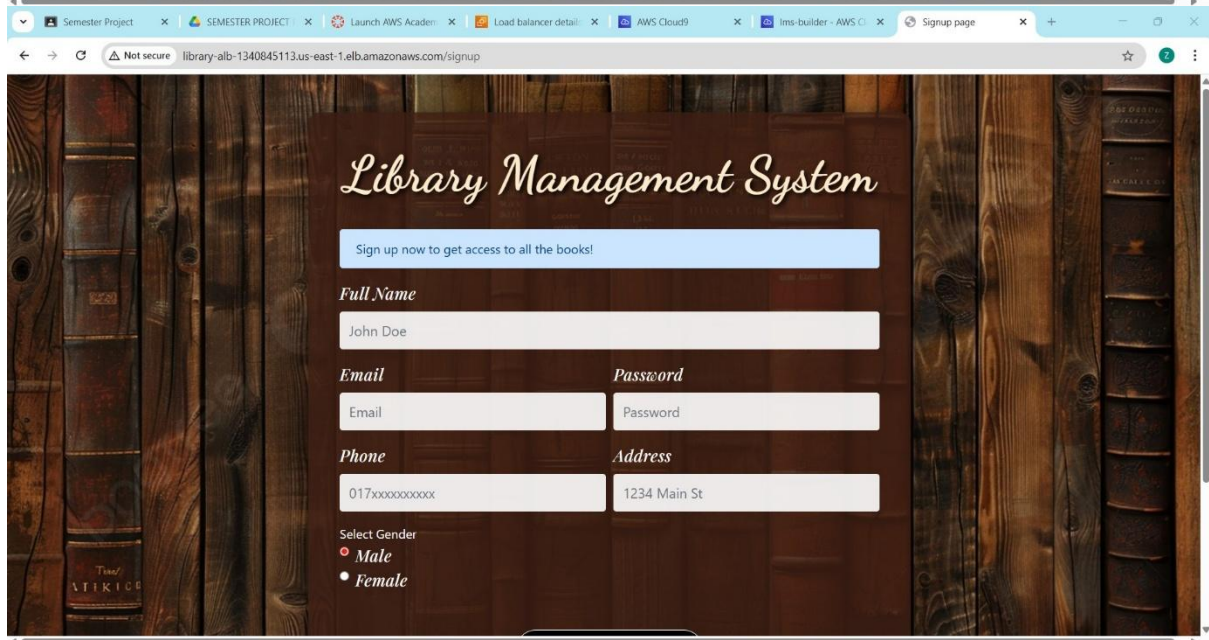
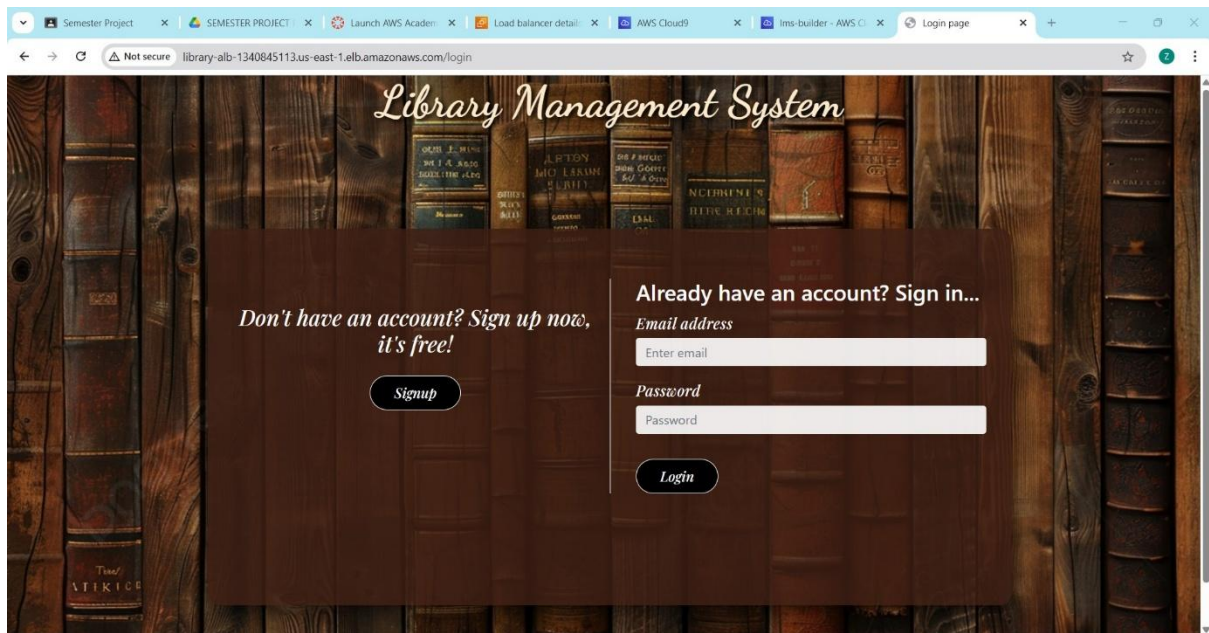
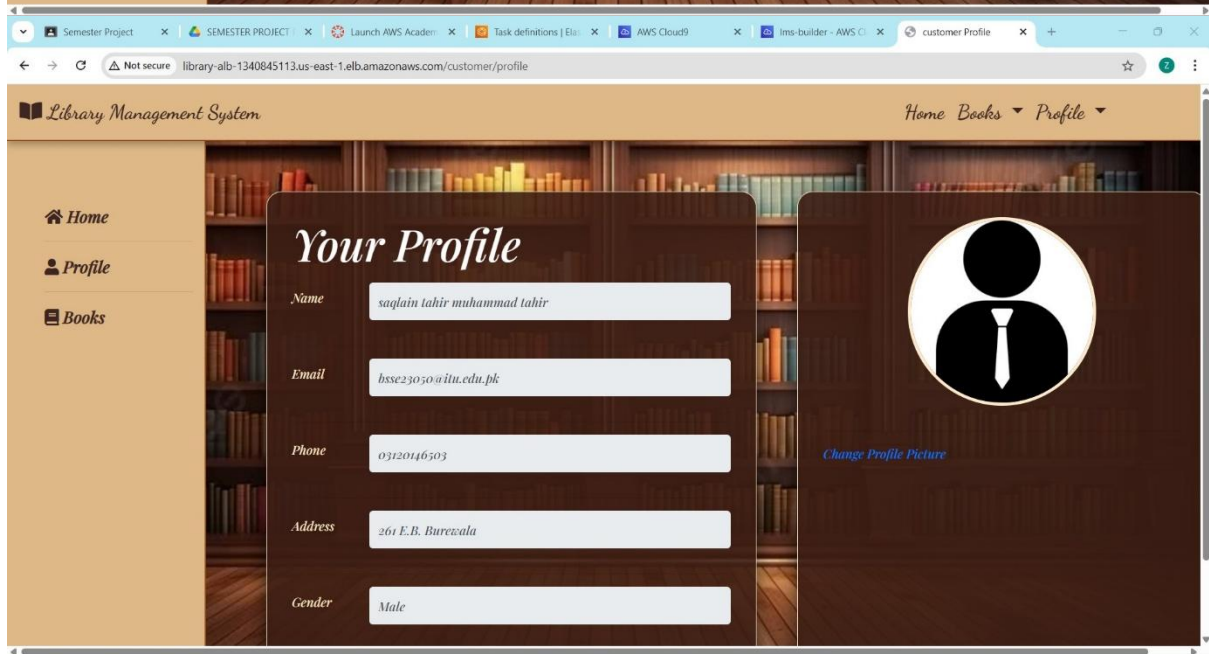
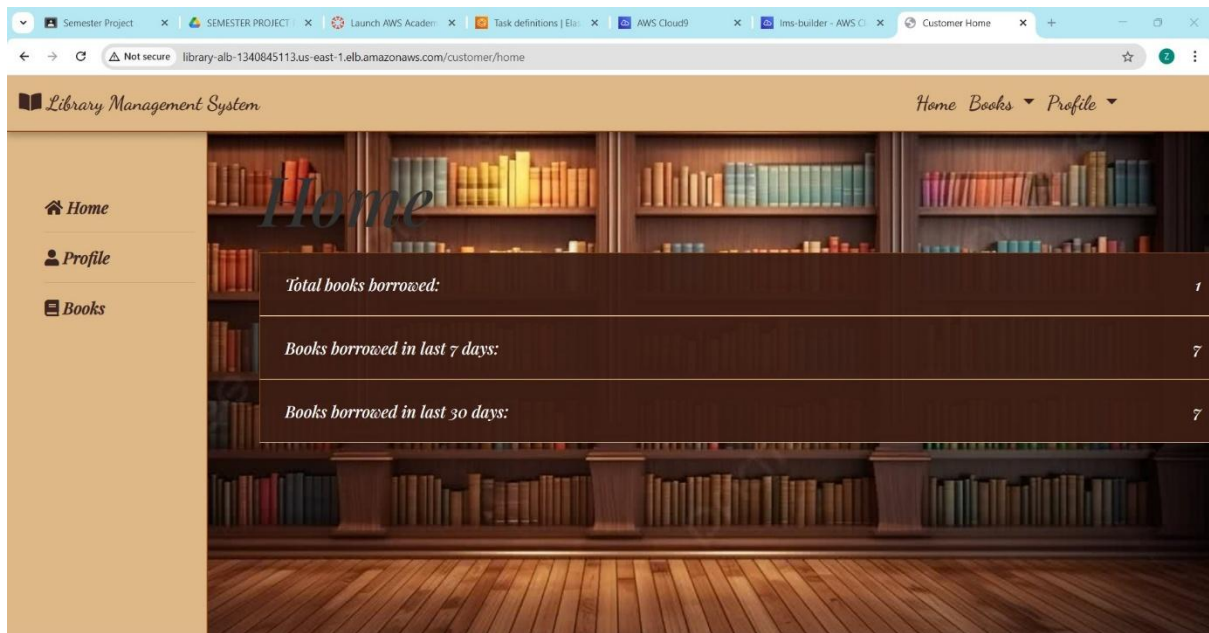


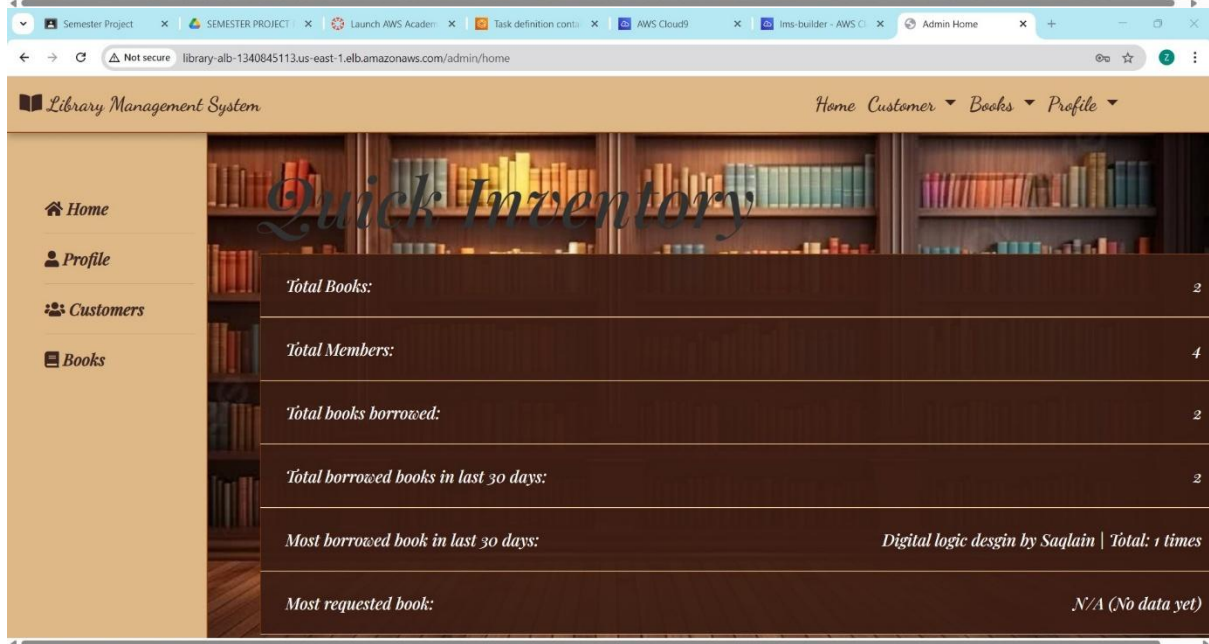
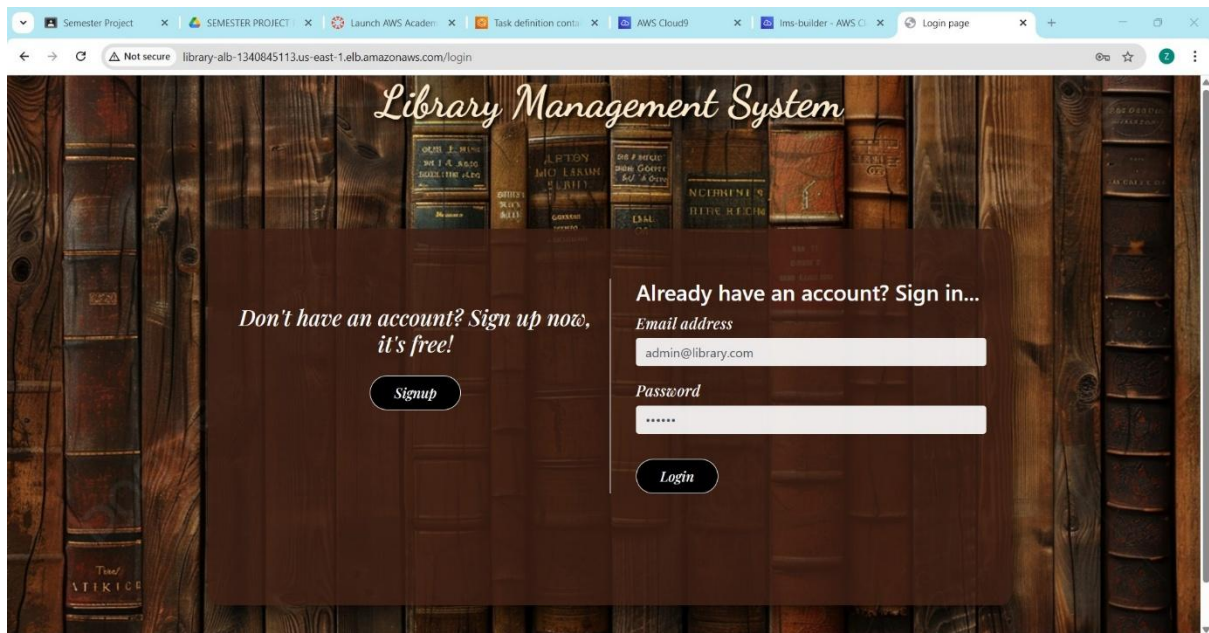
Figure 36: Task Definition Blueprint

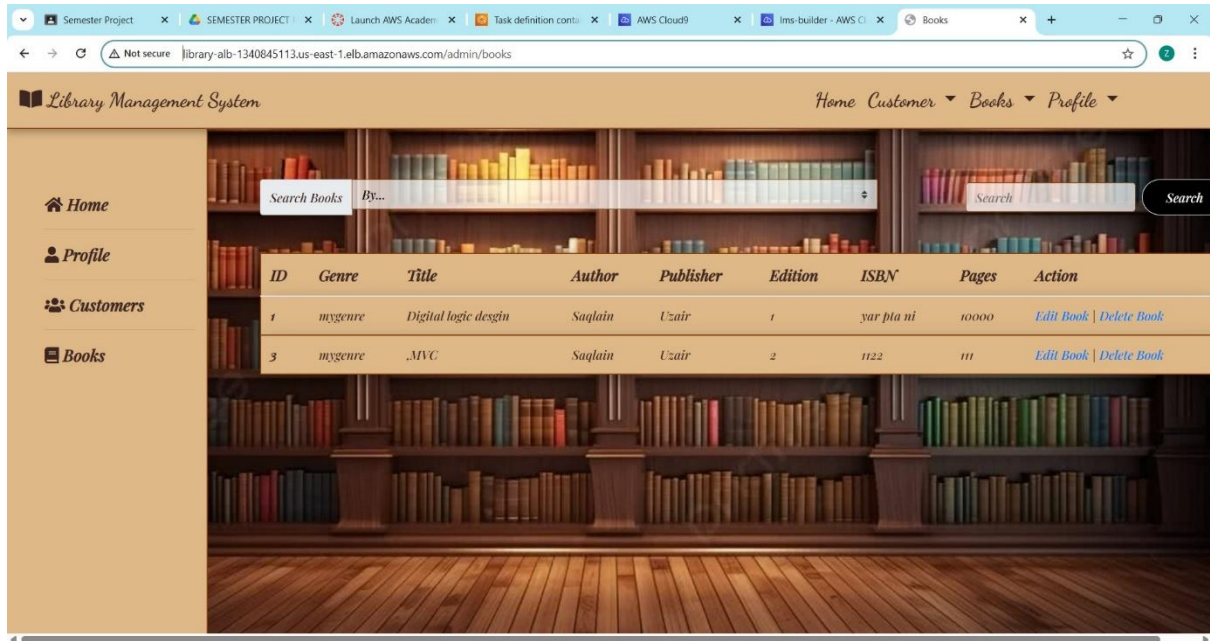
- **Description:** The Task Definition details. It shows the container definitions, including the image mapping and the environment variables (DB_HOST, DB_USER) passed to the container.

PHASE 9: FINAL APPLICATION OUTPUT









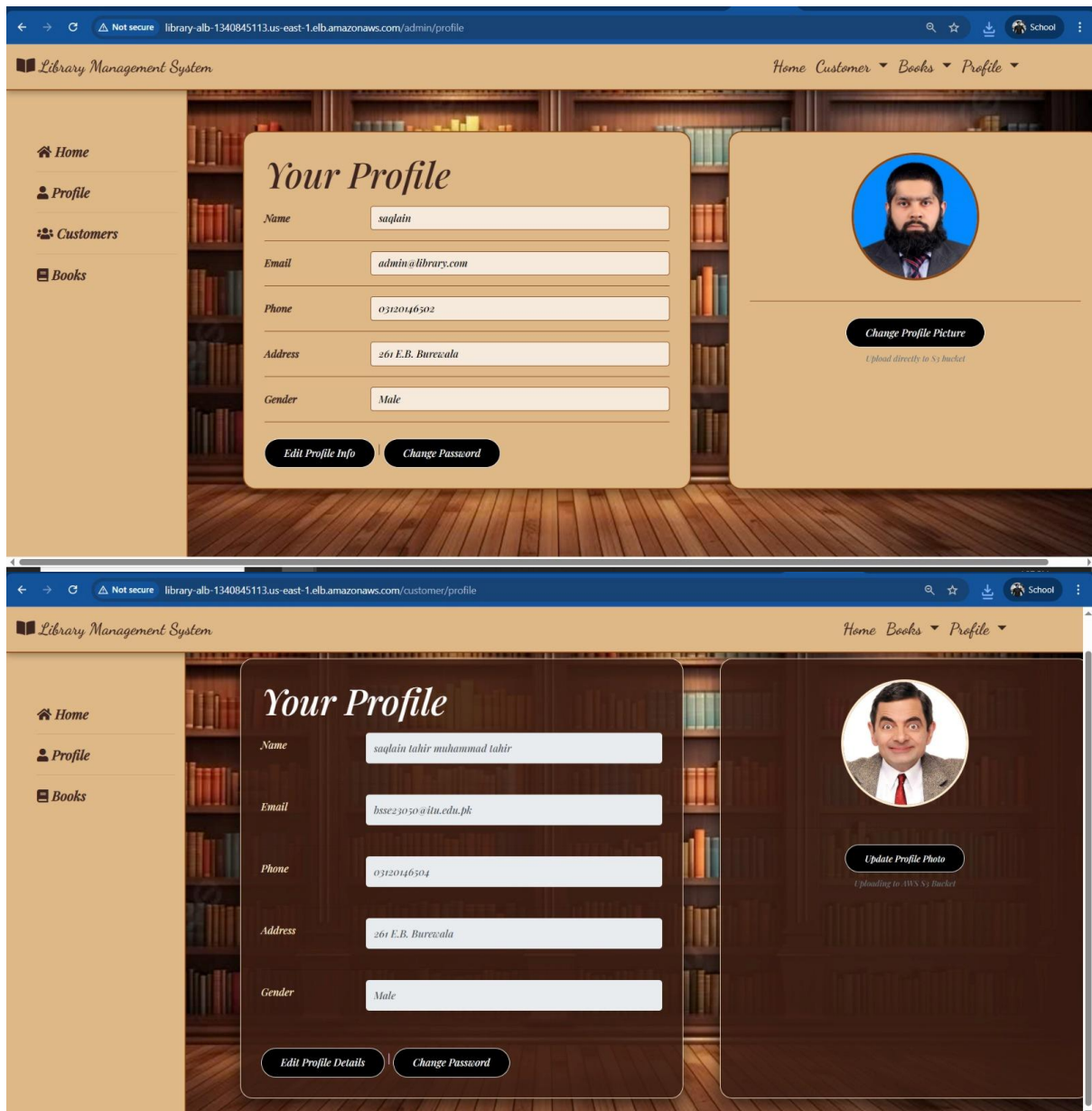
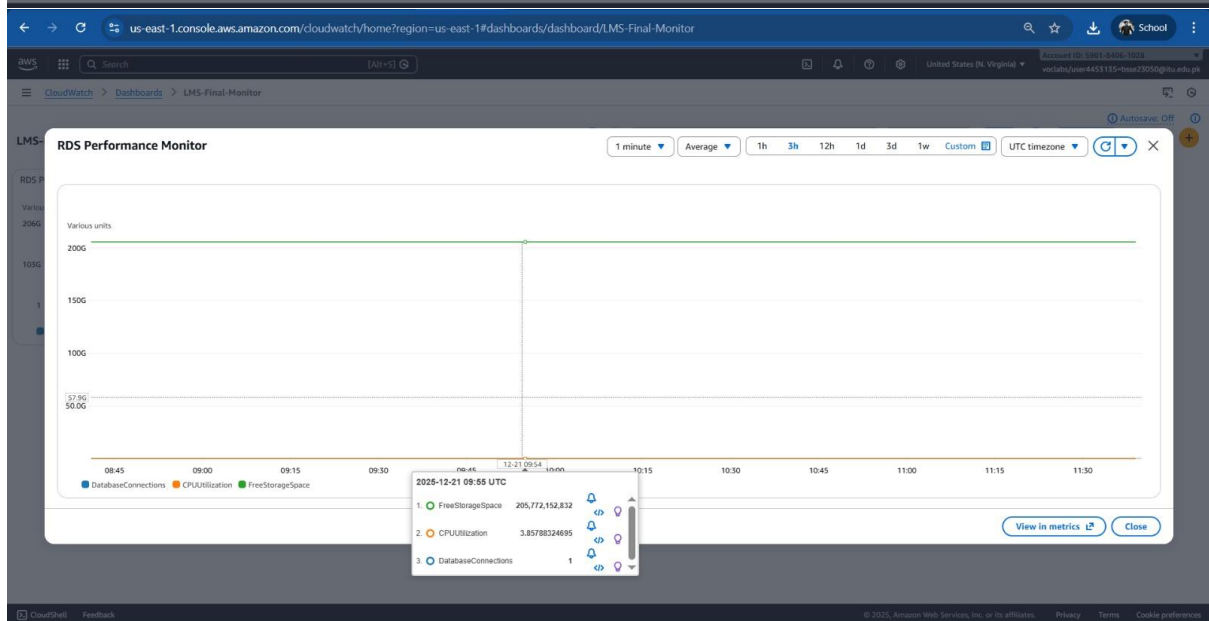
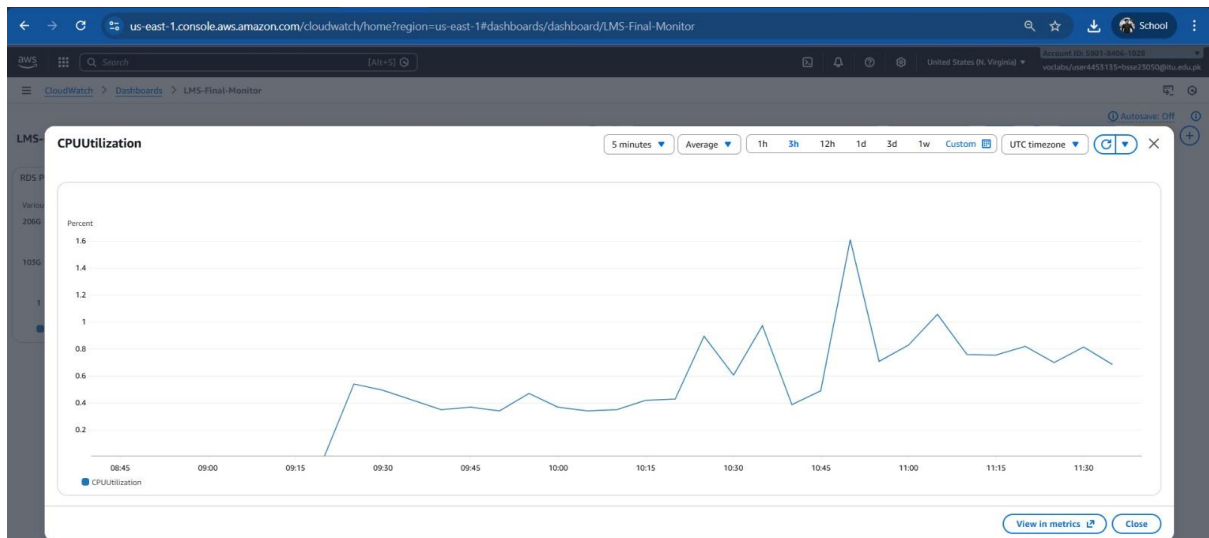


Figure 37-45: Frontend

PHASE 10: MONITORING & OBSERVABILITY



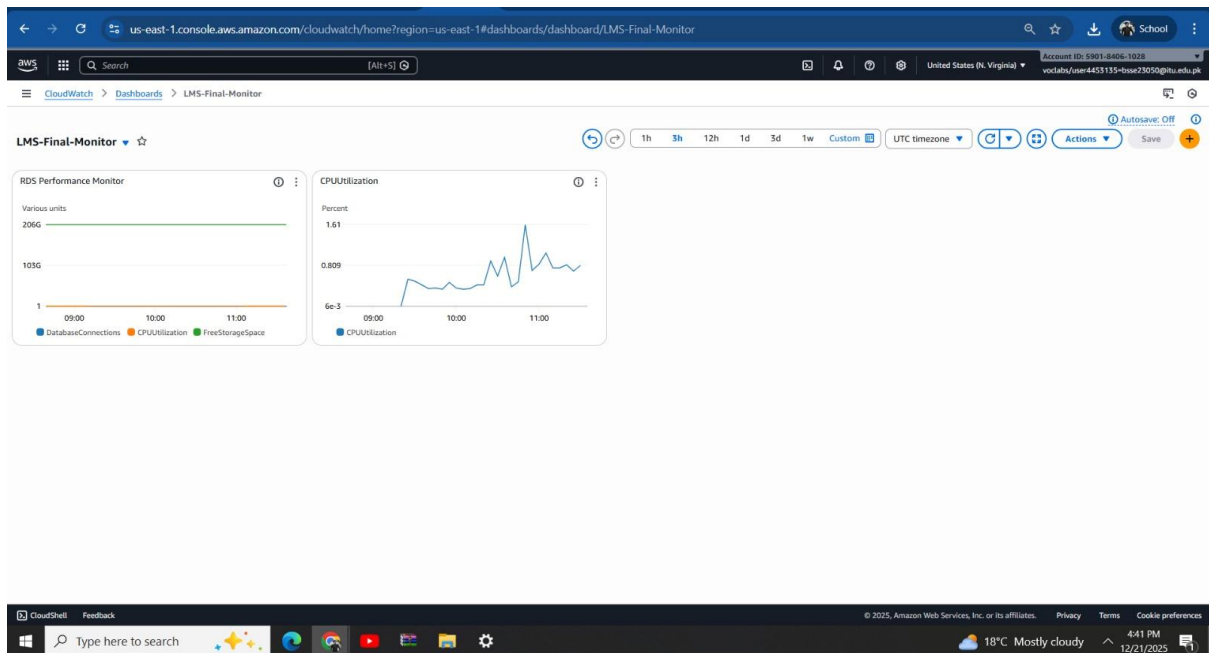


Figure 46-48: CloudWatch RDS Performance and CPU Utilization

- **Description:** A CloudWatch metrics view displaying the performance of the Amazon RDS database instance. The graph specifically shows CPU Utilization percentage over time, indicating the processing load on the database server.

3. REFERENCES

- Amazon Web Services. (2025). *AWS Learner Lab Guide*. AWS Academy.
- Amazon Web Services. (n.d.). *Amazon ECS Documentation*. Retrieved from <https://docs.aws.amazon.com/ecs/>
- GitHub Repo URL: <https://github.com/BSSE-23050/LMS-Project.git>
- Amazon Web Services. (2025). *Amazon ECS Developer Guide*. AWS Documentation.
- Docker Inc. (2024). *Containerizing Python Applications*. Docker Docs.
- Grinberg, M. (2018). *Flask Web Development*. O'Reilly Media.
- AWS Academy. (2024). *Learner Lab Environment & Troubleshooting Guide*.